

Situational Restaurant Recommendation via Web Crawling and LLM-Driven Review Analysis

상황 맞춤형 맛집 추천 : 네이버 지도 웹 크롤링과 LLM 기반 리뷰 분석 활용

“Where2Eat”

Doyoon Kim Hyomin Kim Seohyeon Lee
Seyeon Lee Ryeongah Lim Hannah Jang Kaeun Han



목차

01 Project Workflow

02 Problem definition and necessity

03 Data collection

04 Data preprocessing and storage

05 Data analysis (ML)

**- LLM-based summarization &
situation generation**

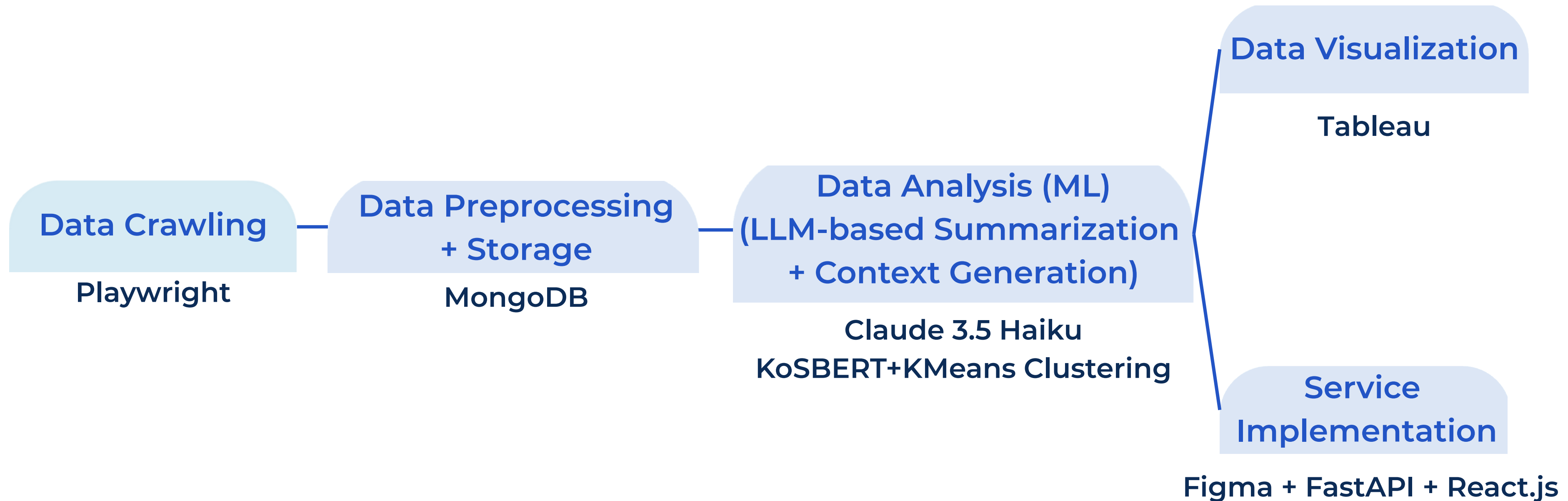
06 Data visualization

07 Service implementation

01

Project Workflow

- **Entire Process**



- **Role**

Data Collection (Crawling)

Doyoon Seohyeon (main)
Hyomin Seyeon Ryeongah
Hannah Kaeun

Data preprocessing + Storage

Ryeongah Hannah Kaeun

Data Analysis (LLM-based)

Hyomin Seyeon Kaeun

Data Visualization

Ryeongah Hannah

Service Implementation

Design : Hannah Kaeun
Development :
Doyoon Seohyeon

Schedule

3월						
일	월	화	수	목	금	토
16	17	18	19	20	21	22
			회의 1			
23	24	25	26	27	28	29
			회의 2 주제 선정	각자 역할 결정		각자 역할 활용 기술 조사
30	31					
4월						
일	월	화	수	목	금	토
		1	2	3	4	5
		1번과정 마감		2번과정 마감	피드백 1	
6	7	8	9	10	11	12
3번과정 마감 / 1차(테스트) 회의				1번 과정 마감	서비스 구현 방식 회의	
13	14	15	16	17	18	19
2번 과정 마감					피드백 2	
20	21	22	23	24	25	26
3번과정 마감 / 2차 (전체) 회의						
27	28	29	30			
사이트 기획 마감	사이트 제작		회의3 2차 역할 분담			

5월						
일	월	화	수	목	금	토
				1	2	3
4	5	6				
시각화, 사이트, 피피티 완성 회의		제출 마감				

• Project Execution with Notion

☰

리스트

📅

표

+

자료실

📄

Streamlit 참고 자료

📄

깃허브 사용법 & 협업하기

📄

서현) 레퍼런스 북마크용

📄

크롤링 실행하기

📄

크롤링 진행 분담

📄

공용 아이디 정보

📄

크롤링 실행하기

☑

회의 유형

Aa

이름

📄

시험기간 수합

📄

주제 제안서

📄

0319 주제 아이디어이션 회의

📄

주제 실효성 검토 및 제안

📄

0326 주제 결정 회의

📄

프로젝트 구체화

각자 맡은 부분에 대해서 조사해, 팀별로 담당할 기술 결정 및 공부

📄

[데이터 수집] 회의

📄

[데이터 수집] 보고

📄

[데이터 전처리 및 적재] 보고

📄

[데이터 분석] 보고

📄

[데이터 전처리 및 적재] 회의

📄

1차

📄

1차

💬 1

☑

회의 유형

Aa

이름

📄

전처리&D...

📄

시각화&개...

📄

크롤링팀

📄

전처리&D...

📄

피드백

📄

피드백

📄

전체 회의

📄

전처리&D...

📄

분석팀

📄

크롤링팀

📄

전체 회의

📄

피드백

📄

크롤링팀

📄

전체 회의

📄

분석팀

📄

전처리&D...

📄

피드백

☑

회의 유형

Aa

이름

📄

MongoDB Atlas 사용법

📄

1차

📄

2차

📄

2차

📄

TA 세션 1

📄

TA 1 Session 피드백

📄

0404 전체 회의

📄

1차 팔로업 문서

📄

1차 팔로업 문서

📄

1차 팔로업 문서

📄

0406 전체 회의

📄

0406 팀별 안건 정리

📄

S3에 저장

📄

0412 전체 회의

📄

2차

📄

2차 팔로업 문서

📄

TA 2 (팀원용)

📄

분석팀

📄

분석팀

📄

분석팀

📄

시각화&개...

📄

시각화&개...

📄

분석팀

📄

전체 회의

📄

시각화&개...

📄

시각화&개...

📄

분석팀

📄

전체 회의

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

시각화&개...

📄

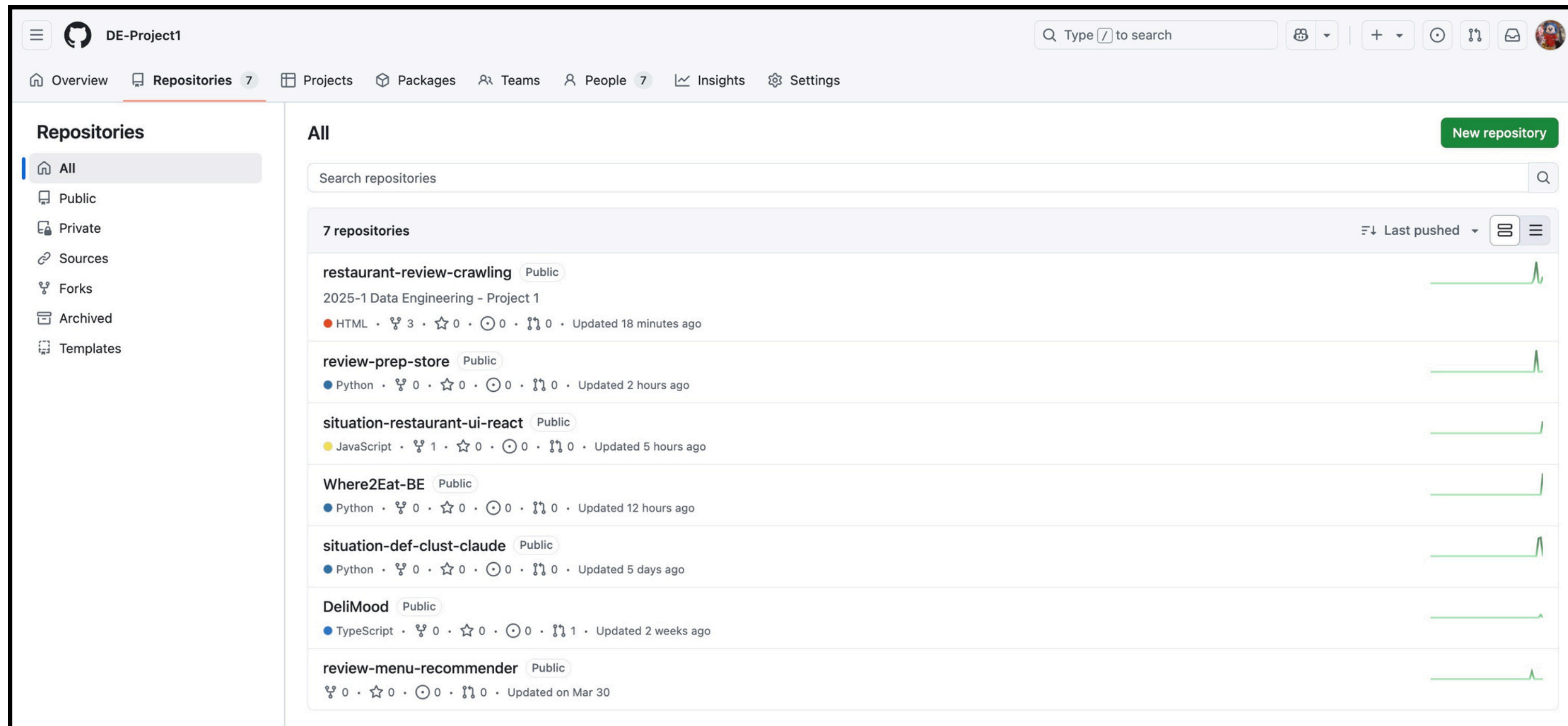
시각화&개...

📄

시각화&개...

• Project Execution with Github

Organization : <https://github.com/DE-Project1>



02

Problem definition and necessity

- “Where2Eat”

soup?



It's raining,
• so I'm craving some warm soup...
What should I search for?

Where to eat
on a rainy day?

It's a special occasion,
• so I'm looking for a restaurant with a
quiet and comfy atmosphere.

- “Where2Eat”

Keyword-based restaurant recommendation services are already saturated.



Users want personalized recommendations that match their “context”, not just simple keywords.



What if I could find restaurants that match my specific situation?

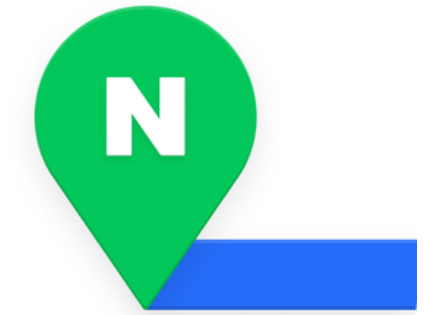
Let's analyze **review data** from each restaurant to derive **the situations** in which people visit!

03

Data collection

• Crawling Overview

Crawling target site



- Crawl restaurants registered on [Naver Map](#)
- PC Naver Map search page, mobile Naver Place detail page

Objective

- Collect "comprehensive restaurant information and reviews by district in Seoul"
- Seoul administrative districts: 425 / ≥ 25 restaurants per district / 100 reviews per restaurant
- [Total crawl of 1+ million](#) restaurant review data

• Crawling Overview

Crawling tool : Playwright



- Supports Chromium (browser for crawling)
- Python language support
- Automatically reconnect when network issues occur
- Dynamic crawler
 - Naver Map is dynamically loaded inside an iframe after JavaScript rendering, making it impossible to access with static crawlers (requests, BeautifulSoup..)
- Asynchronous crawling capable
 - Dramatically reduces crawling time

• Crawling Overview

Development and progress of crawling application

- Used Python + **Playwright v1.51.0**
- Used **uv** as package management tool
- Collected **html (Raw Data)** and **json (restaurant & review data)**
- Saved crawling results locally → **uploaded** to Google Drive
- 7 team members distributed the work
 - 'Divided by 'district' / Each person handled 3~4 districts
 - 1~2 days per person required (total 10~14 days worth)

• Crawling Restaurant Conditions

Search keywords

Search using

'distinct + neighborhood + restaurant'

ex) Mapo-gu Yeonnam-dong restaurant →

To optimize search results

Categories

Based on Naver Smart Place category list

Create a list of **'allowed categories'**

ex. Exclude cafes and bars

```
ACCEPTED_CATEGORIES = [
    # 한식 계열
    "한식", "쌈밥", "보리밥", "비빔밥", "국밥", "찌개, 전골", "곰탕, 설렁탕", "김치찌개", "부대찌개", "청국장",
    "추어탕", "감자탕", "해장국", "갈비탕", "두부요리", "칼국수, 만두", "전, 빈대떡", "백반, 가정식", "한식뷔페", "한정식",
    "국수", "냉면", "막국수", "삼계탕", "주꾸미요리", "순대, 순댓국", "백숙, 삼계탕", "치킨, 닭강정", "생선구이",
    # 고기 계열
    "육류, 고기요리", "닭발", "고기뷔페", "돼지고기구이", "족발, 보쌈", "곰창, 막창, 양", "고기요리", "소고기구이", "닭요리",
    # 분식 계열
    "딤섬, 중식만두", "중화분식", "김밥", "떡볶이", "라면", "만두", "주먹밥", "분식",
    # 일식 계열
    "일식당", "돈가스", "샤브샤브", "오니기리", "오므라이스", "초밥, 롤", "우동, 소바", "일분식라면",
    "일식튀김, 고치", "카레", "일식", "이자카야", "참치회", "회전초밥",
```

Review Count

≥140 visitor reviews

**≥100 reviews among publicly
available ones**

→ To ensure reliable sample data

Rating

≥4.1 stars if provided

→ To ensure restaurant quality

• Crawling Essentials

Restaurant Data

- Place id
- Business name
- Category
- Address
- Business hours
- Service offered
- Rating
- Visitor review count
- Blog review count
- Badge



• Crawling Essentials

Review Data

- Place id
- Nickname
- Content
- Review date
- Visit situation keywords
- Restaurant review keywords
- Total review count by author
- Visit frequency to the restaurant



• Crawling Pipeline

1. Confirm and input administrative distinct codes

adm_dong_list.csv, target_adm_dong_codes.txt

2. When crawling code runs, the following steps execute

main.py

a. Search “OO Distinct OO Neighborhood Restaurant” on Naver Map → Collect place IDs matching conditions

crawler_pipeline.py, place_searcher.py

b. Access detail page with place ID → Crawl restaurant info and reviews

place_data_collector.py

c. Save collected restaurant and review data as json

save_data.py, json_data/place_info folder, json_data/reviews folder

d. Save raw data per restaurant as html

save_data.py, raw_data folder

• Challenges

Lazy loading

Lazy Loading
: Method of loading
only content needed
by users first

- When searching 'restaurants by neighborhood', search results don't load all at once
- Modified code to implement automatic scrolling

Home tab

Some information not
displayed on restaurant's
home tab page

- Toggle collapsed elements before crawling



Crawling time

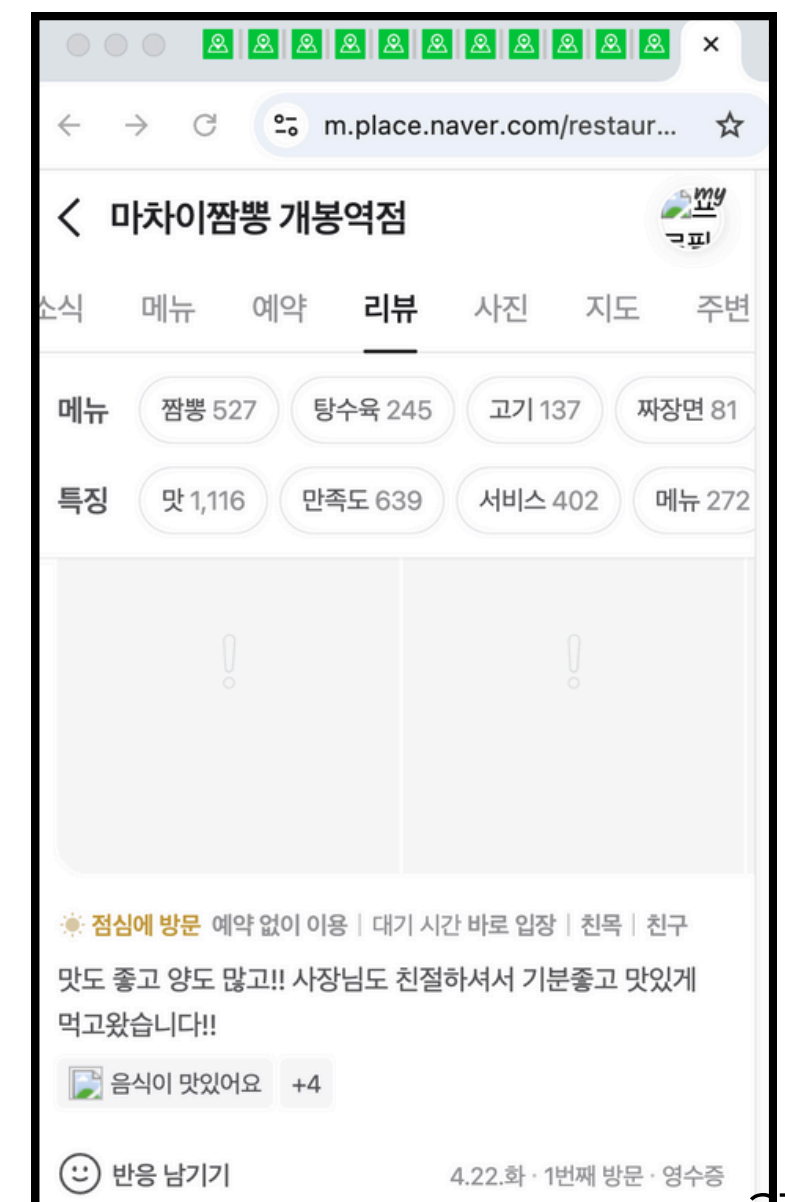
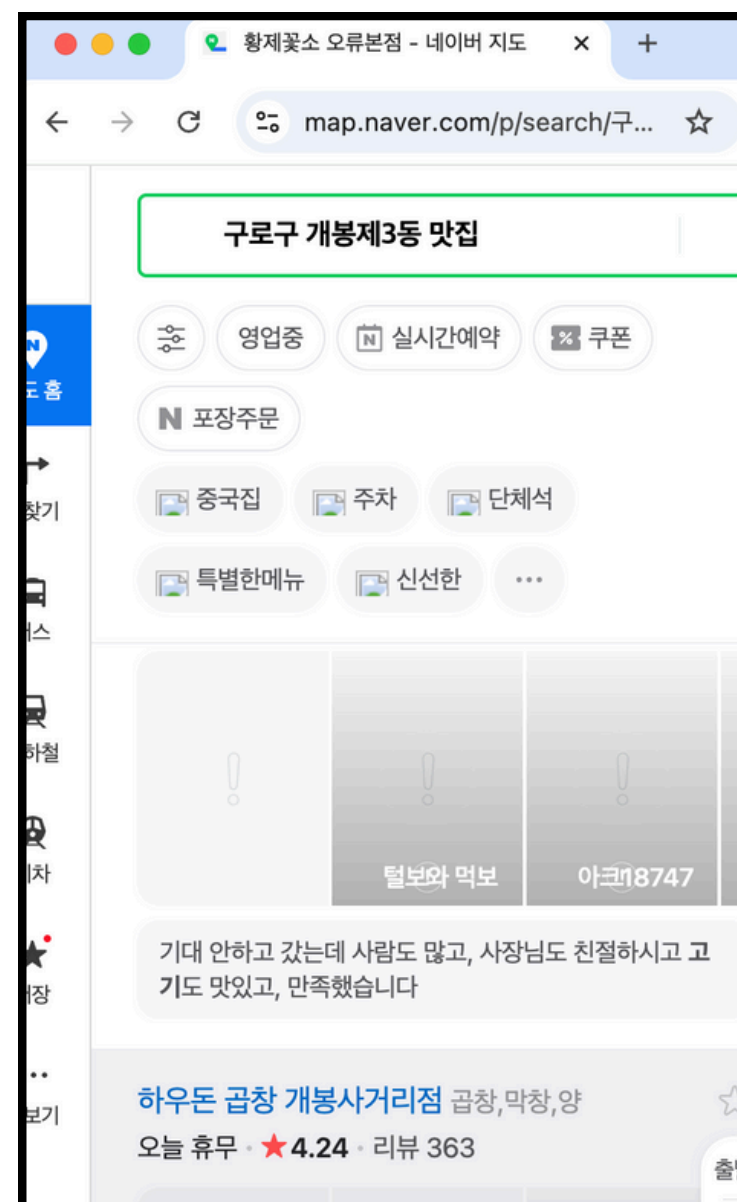
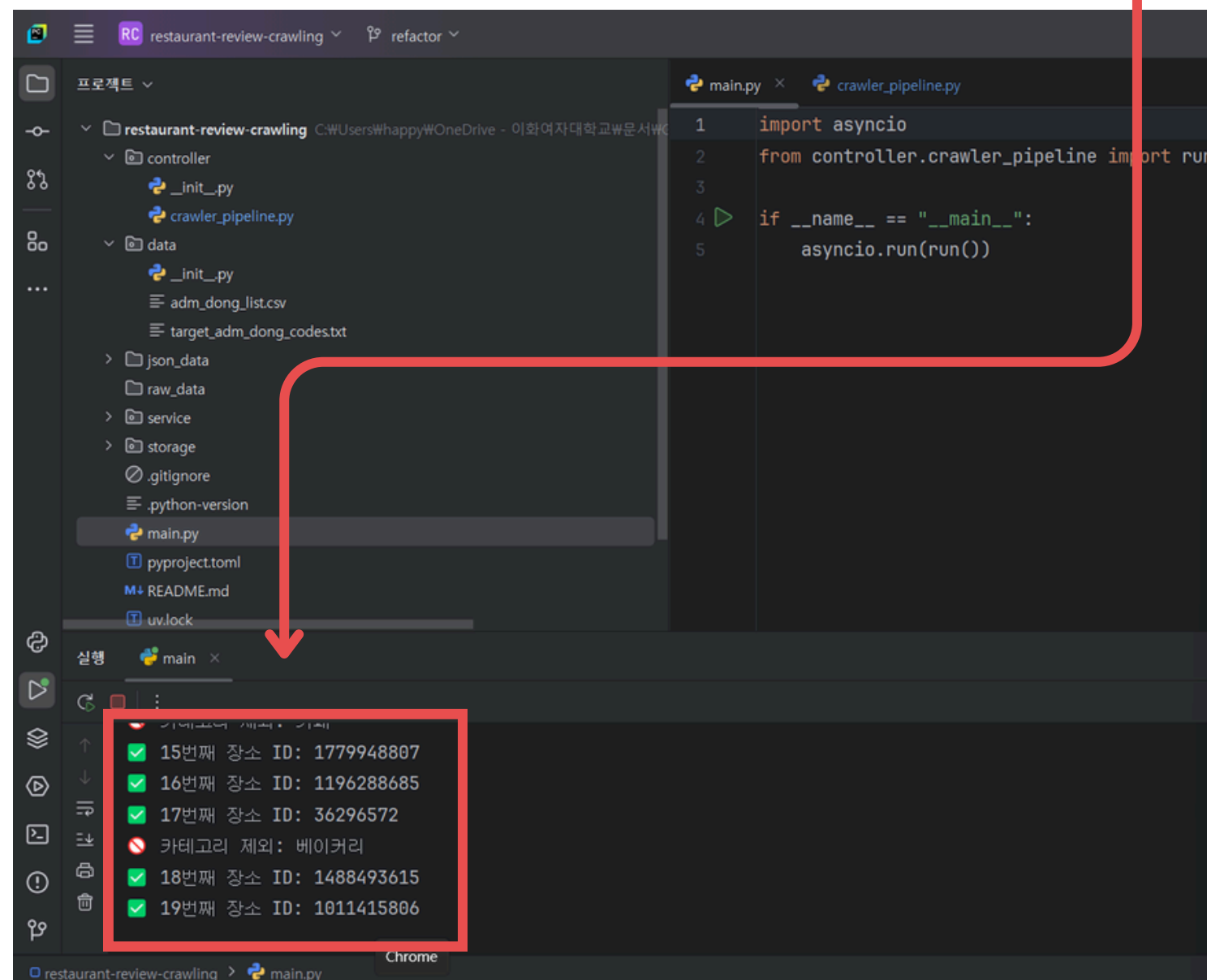
Crawling took too long

- Implemented asynchronous crawling using Python's **asyncio** library and Playwright's **async_playwright**

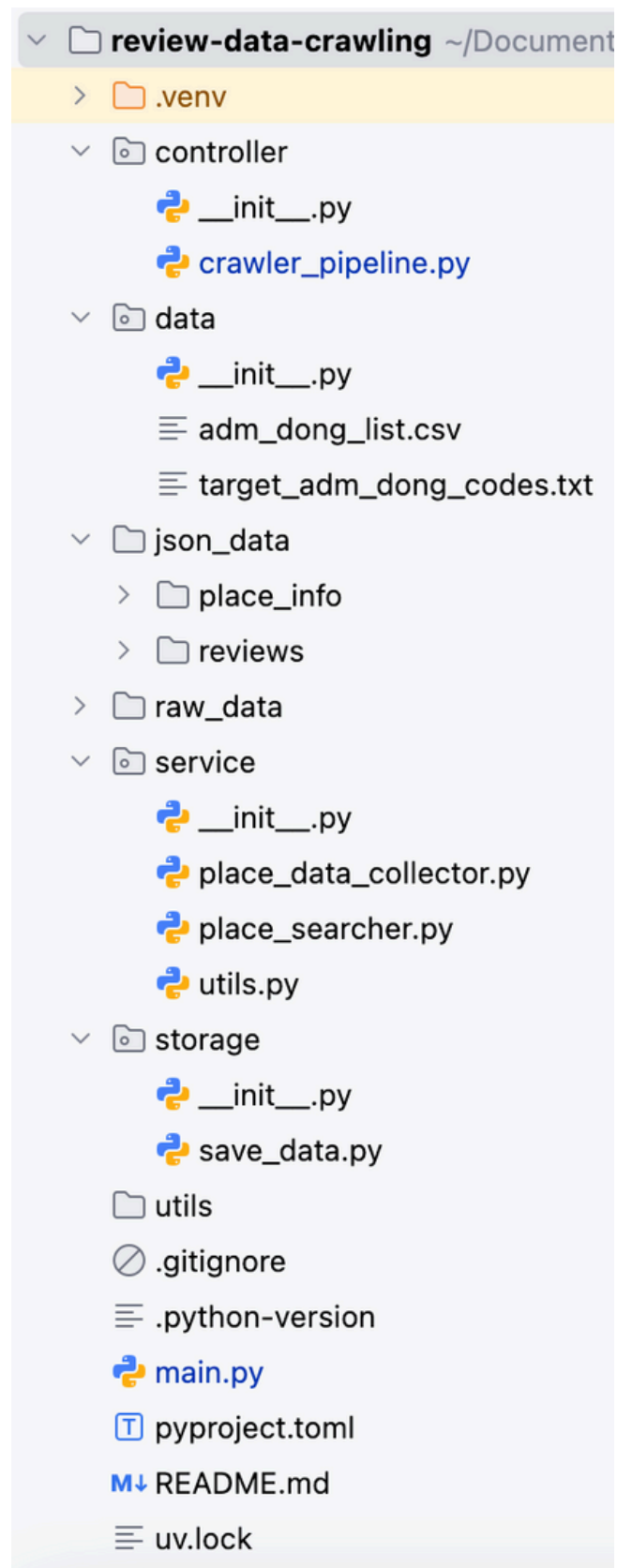
- Improved speed by handling I/O wait time and running multiple tabs simultaneously

• Crawling Execution Process

- Crawling screen when running `main.py`
 - Crawling executes as crawler window appears
1. Restaurant filtering, ID collection
Verify filtering in console
collect place IDs matching conditions
 2. Asynchronous crawling
→ Collect restaurants, reviews, raw data
→ 4 restaurants * 3 tabs = 12 tabs



• Crawling Code Description



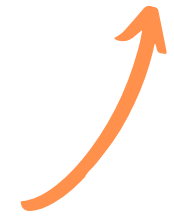
- Processes in order

• data/

- **adm_dong_list.csv**: File containing Seoul administrative distinct codes, city, district, and neighborhood information
- **target_adm_dong_codes.txt**: Files for inputting administrative district codes



	adm_dong_code	city	district	neighborhood
1	1111051500	서울특별시	종로구	청운효자동
2	1111053000	서울특별시	종로구	사직동
3	1111054000	서울특별시	종로구	삼청동
4	1111055000	서울특별시	종로구	부암동
5	1111056000	서울특별시	종로구	평창동
6	1111057000	서울특별시	종로구	무악동



1	1120054000
2	1120055000
3	1120061500
4	1120062000
5	1120064500
6	1120066000

• **main.py**: Execute crawling pipeline

• controller/

- **crawler_pipeline.py**: Complete crawling pipeline code (targeted search for administrative districts and individual restaurant. Individual restaurant crawling done asynchronously in group of four.)

• service/

- **place_searcher.py**: Crawl restaurants matching conditions from search page
- **place_data_collector.py**: Crawl food, info, and review tab from Naver Place
- **utils.py**: File containing utility methods

• storage/**save_data.py**: json files loading/saving and saving them as html files

• json_data/

- **place_info/**: Directory where crawled restaurant place info JSON files are stored
- **reviews/**: Directory where crawled review JSON files are stored

• **raw_data/**: Directory where completed HTML fiels of crawled data per restaurant are stored



```

1 import asyncio
2 from controller.crawler_pipeline import run
3
4 if __name__ == "__main__":
5     asyncio.run(run())
  
```


• Crawling Code Description

main.py > **run()**

crawling_pipeline.py

run() : Method that reads input and executes crawling by administrative district

- Load target administrative district codes to crawl (from `data/target_adm_dong_codes.txt`)
- Open file containing administrative district info (`data/adm_dong_list.csv`) and extract district, neighborhood for each district code
- Proceed with each `adm_dong_code` as follows:

1. Collect `place_id` list from `search_and_fetch_place_ids`("OO District OO Neighborhood", `MAX_PLACES`)

2. Execute individual place data crawling from `crawl_from_place_ids(place_ids, adm_dong_code)`

```
import csv
import asyncio
import os
import time
from playwright.async_api import async_playwright
from service.place_searcher import search_and_fetch_place_ids
from service.place_data_collector import fetch_home_page_and_get_place_info, fetch_review_page_and_get_reviews, fetch_info_page
from service.utils import block_unnecessary_resources, log
from storage.save_data import save_place_info_json, save_reviews_json, save_place_raw_html, save_failed_places_json

RAW_DIR="raw_data"
TARGET_TXT_PATH="data/target_adm_dong_codes.txt" # 크롤링할 행정동코드
ADM_DONG_CSV_PATH="data/adm_dong_list.csv" # 행정동 csv 파일
MAX_PLACES=60

# 메인 실행 함수
async def run() -> None:
    # 파일을 입력으로 대상 행정동코드를 읽어옴
    with open(TARGET_TXT_PATH, 'r', encoding='utf-8-sig') as f:
        target_codes = set(line.strip() for line in f if line.strip())

    # 행정동코드로 구+동 정보 획득
    adm_dong_dict = {}
    with open(ADM_DONG_CSV_PATH, newline='', encoding='utf-8-sig') as csvfile:
        reader = csv.DictReader(csvfile)
        for row in reader:
            adm_dong_code = row["adm_dong_code"]
            if adm_dong_code in target_codes:
                # "구 동" 형태로 value 구성
                adm_dong_dict[adm_dong_code] = f"{row['district']} {row['neighborhood']}"

    for adm_dong_code in target_codes:
        global_start = time.time()
        # 크롤링 실행 (실패 후 중복 확인 로직 추가)
        try:
            place_ids = await search_and_fetch_place_ids(adm_dong_dict[adm_dong_code], MAX_PLACES) # 최대 수집 장소 수
            print(f"{adm_dong_code}: {len(place_ids)}개 수집 시작")
            await crawl_from_place_ids(list(place_ids), adm_dong_code)

        except Exception as e:
            print(f"❌ [ERROR] 크롤링 중 오류 발생 - {e}")

    global_elapsed = time.time() - global_start
    print(f"✅ 전체 소요시간: {global_elapsed:.1f} sec\n")
```

• Crawling Code Description

run() > 1. Collect Place ID list

place_searcher.py

search_and_fetch_place_ids (district_neighborhood, max_places)

: Method that searches restaurant recommendations in the neighborhood on Naver Map and finds up to max_places restaurant IDs that meet the conditions

- URL: https://map.naver.com/p/search/{district_neighborhood}%20맛집
- Set up browser(Chromium), context, page
- Call block_unnecessary_resources defined in utils.py → Blocks image, stylesheet, and font requests on the page, preventing unnecessary resource loading and improving crawling speed.
- **Search results load dynamically : iframe#searchIframe**
 - Upon first page entry, scroll down div#_pcmap_list_scroll_container to load all elements, then scroll back to top for individual element access
 - In [parse_places_from_items](#), add place IDs to list only if they meet restaurant conditions
- **Navigate search result pages:**
 - Find and click a.eUTV2:has(span.place_blind:text("next page"))

```
# 장소 검색 및 ID 리스트 크롤링 함수
# 특정 지역에서 조건을 만족하는 장소 최대 max_places개까지 수집 (place_id만)
async def search_and_fetch_place_ids(district_neighborhood: str, max_places: int) -> List[str]:
    """ 2개의 사용 위치 ㄹ Seohyun Lee +1 """
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=False, args=["--window-size=400,800"])
        context = await browser.new_context(
            viewport={"width": 400, "height": 800},
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
            "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36"
        )
        page = await context.new_page()
        await block_unnecessary_resources(page)
        # 페이지 이동
        search_word = district_neighborhood + " 맛집"
        await page.goto(f"https://map.naver.com/p/search/{search_word}")
        await page.wait_for_timeout(2000) # 초기 로딩 대기
        # place_ids에 장소 ID 수집
        place_ids = []
        current_page = 1
        while current_page <= 5 and len(place_ids) < max_places: # 최대 페이지 번호 5; 장소 수 max_places 이하여야
            try:
                print(f"\n== {current_page}페이지 크롤링 시작 ==")
                iframe_element = await page.wait_for_selector("iframe#searchIframe", timeout=5000)
                search_frame = await iframe_element.content_frame()
                scroll_container = await search_frame.wait_for_selector("div#_pcmap_list_scroll_container", timeout=10000)
                await scroll_until_no_more(scroll_container) # 스크롤 다운로드 전체 아이템 로딩
                place_items = await search_frame.query_selector_all("li.UeZoS.rTjJo")
                print(f"✅ {current_page}페이지에서 {len(place_items)}개 장소 발견")
                # place_items에서 장소 new_ids 얻어 place_ids에 추가
                await parse_places_from_items(place_items, page, max_places, place_ids)
                if len(place_ids) >= max_places:
                    break
                # 다음 페이지 넘기기
                next_btn = search_frame.locator('a.eUTV2:has(span.place_blind:text("다음페이지"))').first
                if await next_btn.count() == 0:
                    print("❌ 다음페이지 버튼 못 찾음. 종료")
                    break
                # 비활성화 상태인지 확인
                if await next_btn.get_attribute("aria-disabled") == "true":
                    print("⚠️ 다음 페이지 버튼이 비활성화됨. 종료")
                    break
                # 클릭 후 대기
                current_page += 1
                await next_btn.click()
                await page.wait_for_timeout(1500)
            except Exception as e:
                print(f"[ERROR] 페이지 처리 중 오류: {e}")
                break
        await context.close()
        await browser.close()
        return place_ids
```

• Crawling Code Description

run() > 1. Collect Place ID list

place_searcher.py

parse_places_from_items (items, page, max_places, place_ids)

: Method that adds place IDs to list if the place meets restaurant conditions

- **Category check:** span.KCMnt → Verfiy inclusion in ACCEPTED_CATEGORIES
- **Visitor review count check:** span.h69bs → Parse number from "review" text → ≥ 140
- **Rating check:** span.h69bs.orXYY → Parse float from rating text if exists → ≥ 4.1
- **Place ID extraction :** Click div.place_bluelink → Page navigation, match page.url with regex /place/(\d+) → Obtain place_id

```
# 수집 대상인 음식점 카테고리 리스트
ACCEPTED_CATEGORIES = [
    # 한식 계열
    "한식", "쌈밥", "보리밥", "비빔밥", "국밥", "짜개, 전골", "곰탕, 설렁탕",
    "추어탕", "감자탕", "해장국", "갈비탕", "두부요리", "칼국수, 만두", "국수",
    "냉면", "막국수", "삼계탕", "주꾸미요리", "순대, 순댓국", "백종원",
    # 고기 계열
    "육류, 고기요리", "닭발", "고기뭇배", "돼지고기구이", "족발, 보쌈", "곱창",
    # 분식 계열
    "떡볶이, 떡갈떡", "김밥", "떡볶이", "라면", "만두", "주먹밥",
    # 일식 계열
    "일식", "돈가스", "샤브샤브", "오니기리", "오므라이스", "초밥, 롤",
    "일식튀김, 고치", "카레", "일식", "이자카야", "참치회", "회전초밥",
    # 중식 계열
    "중식", "양꼬치", "마라탕", "계요리", "닭집", "중식", "중화요리",
    # 양식 계열
    "브런치", "스테이크, 힙", "피자", "파스타", "패밀리레스토랑",
    "경양식", "스파게티", "브런치카페", "양식", "독일음식", "스파게티, 파스타",
    # 세계 음식
    "이탈리아음식", "스페인음식", "프랑스음식", "터키음식", "아프리카음식",
    "멕시코, 남미음식", "베트남음식", "태국음식", "인도음식", "아시아음식",
    "멕시코, 브라질", "퓨전음식",
    # 주점
    "요리주점", "술집", "포장마차", "바(BAR)", "와인바", "와인", "맥주",
    # 그 외 음식점
    "푸드트럭", "푸드코트", "토스트", "햄버거", "죽", "도시락", "샐러드",
    "다이어트, 샐러드", "채식, 샐러드뷔페", "얼밥", "찰판요리", "치킨", "패밀리레스토랑",
    "아식", "사찰, 영양탕", "사찰음식", "기사식당", "황토음식", "이북음식",
    "해물, 생선요리", "일식, 초밥뷔페", "해산물뷔페", "생선회", "도시락, 김밥"
]
```

```
MIN_REVIEW_COUNT = 140 # 방문자 리뷰
MIN_RATING = 4.1 # 최소 별점 기준
```

```
async def parse_places_from_items(items, page, max_places: int, place_ids):
    for item in items:
        if len(place_ids) >= max_places:
            break
        try:
            # 카테고리
            category_el = await item.query_selector("span.KCMnt")
            category = await category_el.text_content() if category_el else "N/A"
            if category not in ACCEPTED_CATEGORIES:
                print(f"❌ 카테고리 제외: {category}")
                continue
            # 방문자 리뷰
            review_el = await item.query_selector_all("span.h69bs")
            review_count = 0
            for span in review_el:
                text = await span.inner_text()
                if "리뷰" in text:
                    digits = re.search(pattern=r"(\d+)", text)
                    if digits:
                        review_count = int(digits.group(1))
                    break
            if review_count < MIN_REVIEW_COUNT:
                print(f"❌ 리뷰 수 부족: {review_count}")
                continue
            # 별점
            rating_el = await item.query_selector("span.h69bs.orXYY")
            rating_text = await rating_el.text_content() if rating_el else None
            if rating_text:
                match_rating = re.search(pattern=r"(\d+)", rating_text)
                rating = float(match_rating.group()) if match_rating else 0.0
                if rating < MIN_RATING:
                    print(f"❌ 별점 낮음: {rating}")
                    continue
            # 상세 페이지 이동 후 ID 추출
            click_target = await item.query_selector("div.place_bluelink")
            if click_target:
                await click_target.click()
                await page.wait_for_timeout(1500)
                detail_url = page.url
                match = re.search(pattern=r'/place/(\d+)', detail_url)
                if match:
                    place_id = match.group(1)
                    print(f"✅ {len(place_ids) + 1}번째 장소 ID: {place_id}")
                    place_ids.append(place_id)
                else:
                    print(f"❌ place_id 추출 실패")
            except Exception as e:
                print(f"[ERROR] 항목 처리 중 오류: {e}")
                continue
```


• Crawling Code Description

run() > 2. Individual place data crawling

crawling_pipeline.py

crawl_from_place_ids (place_ids, adm_dong_code)

: Method for asynchronously crawling
up to 4 places simultaneously

1. Execute asynchronous crawling with async_playwright
2. Create browser, context (viewport, User-Agent settings)
3. Batch 4 place IDs at a time
 - For each of place_ids, create **scrape_place_details(context, place_id, adm_dong_code)** coroutine and add to tasks list (store with exception handling)
 - When await await asyncio.gather(*tasks, return_exceptions=True) is called, 4 coroutines execute asynchronously → Detail page crawling methods for 4 restaurants execute simultaneously

```
# place_ids로부터 음식점을 4개씩 배치 크롤링
async def crawl_from_place_ids(place_ids: list[str], adm_dong_code): 1개의 사용 위치 ㄹ Seohyun Lee
    results = []
    async with async_playwright() as p:
        place_ids = list(place_ids)

        browser = await p.chromium.launch(headless=False)
        context = await browser.new_context(
            viewport={"width": 400, "height": 800},
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like G
        )

        # 4개씩 처리
        for i in range(0, len(place_ids), 4):
            batch = place_ids[i:i + 4]

            tasks = []
            for place_id in batch:
                output_path = f"{RAW_DIR}/adc_{adm_dong_code}_place_rawdata_{place_id}.html"
                if os.path.exists(output_path):
                    print(f"이미 파일 있음 → 스킵 (PlaceID: {place_id})")
                    continue
                tasks.append(scrape_place_details(context, place_id, adm_dong_code))

            # 동시에 실행
            batch_results = await asyncio.gather(*tasks, return_exceptions=True)

            # 결과 저장
            for result in batch_results:
                if isinstance(result, dict):
                    results.append(result)
            await context.close()
            await browser.close()

    return results
```

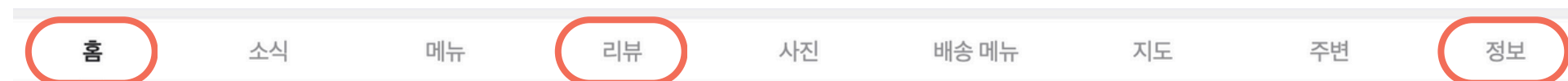

• Crawling Code Description

run() > 2. Individual place data crawling

crawling_pipeline.py

scrape_place_details (context, place_id, adm_dong_code)

Method for collecting detailed Naver Place page data - simultaneously crawl home, review tab, and info tab



1. Create 3 pages from current context
2. Call block_unnecessary_resources for each page → Block loading unnecessary resources
3. Execute asynchronous crawling on 3 pages with asyncio.gather():
 - fetch_home_page_and_get_place_info(home_page, ...)
 - fetch_review_page_and_get_reviews(review_page, ...)
 - fetch_info_page(info_page, ...)
4. Call HTML and JSON save methods:
 - Extract raw HTML using page.content()
 - save_place_info_json, save_reviews_json, save_place_raw_html
5. Close all 3 pages when complete

```
# 식당 1개 크롤링
async def scrape_place_details(context, place_id: str, adm_dong_code: int): 1개의 사용 위치  ⚡ Seohyun Lee
    log("START", place_id)
    start_time = time.time()
    html_data = {"place_id": place_id, "home_html": None, "info_html": None, "reviews_html": None}

    # 탭 열기
    home_page = await context.new_page()
    info_page = await context.new_page()
    review_page = await context.new_page()

    # 리소스 차단
    await block_unnecessary_resources(home_page)
    await block_unnecessary_resources(info_page)
    await block_unnecessary_resources(review_page)

    # 세 가지 작업을 병렬로 실행. 하나라도 예외 발생 시 전체가 예외를 던짐
    results = await asyncio.gather(
        fetch_home_page_and_get_place_info(home_page, place_id, adm_dong_code),
        fetch_review_page_and_get_reviews(review_page, place_id),
        fetch_info_page(info_page, place_id),
        return_exceptions=True
    )
    place_info, reviews, info_html = results
    if isinstance(place_info, Exception) or isinstance(info_html, Exception) or isinstance(reviews, Exception):
        # 하나라도 실패 시 함수 전체 종료
        print(f"❌ {place_id} - 수집 불충분 → 저장 생략")
        await home_page.close()
        await info_page.close()
        await review_page.close()
        return # 함수 전체 종료

    try:
        html_data["home_html"] = await home_page.content()
        html_data["info_html"] = info_html
        html_data["reviews_html"] = await review_page.content()

        save_place_info_json(place_info, adm_dong_code)
        save_reviews_json(reviews, adm_dong_code)
        save_place_raw_html(place_id, adm_dong_code, html_data)

        elapsed = time.time() - start_time
        log("SAVED", place_id, extra=f"{elapsed:.1f} sec")
    except Exception as e:
        save_failed_places_json(place_id, adm_dong_code)
        log("ERROR", place_id, extra=str(e))
    finally: # 페이지 닫기
        await home_page.close()
        await info_page.close()
        await review_page.close()
```

• Crawling Code Description

run() > 2. Individual place data crawling

place_data_collector.py

fetch_home_page_and_get_place_info (page, place_id, adm_dong_code)

: Method for crawling home tab

- URL: `https://m.place.naver.com/restaurant/{place_id}/home`
- Restaruant name : `span.GHAhO`, category: `span.lnJFt`, address: `span.LDgIH`
- Business hours : `a.gKP9i[aria-expanded="false"]` toggle click to load all hours (parse with `parse_opening_hours(page)`)
- Naver rating : `span.PXMot.LXIwF` → parse number from rating text
- Visitor, blog review count : `a[href*="/review/visitor"]`, `a[href*="review/ugc"]`
- Service offered : `div.xPvPE`, Badge : `div.XtBbS`
- Crawling start time : `datetime.now()`

fetch_info_page (page, place_id)

: Method for crawling info tab

- URL: `https://m.place.naver.com/restaurant/{place_id}/information`
- Wait for loading : `page.wait_for_load_state('domcontentloaded')`
- Complete scroll until page changes, then extract initial HTML for conversion

```
import asyncio
import re
from datetime import datetime
import random
from playwright.async_api import Page
from service.utils import safe_page_content

# 장소 상세 정보 수집
# 홈 페이지로 이동
url = f"https://m.place.naver.com/restaurant/{place_id}/home"
await page.goto(url)
await page.wait_for_timeout(1000)
info = {}

try:
    # 상호명, 카테고리, 주소
    name_el = await page.query_selector('span.GHAhO')
    category_el = await page.query_selector('span.lnJFt')
    address_el = await page.query_selector('span.LDgIH')
    await page.wait_for_timeout(random.randint(a=1000, b=1500))
    # 영업시간 클릭해 펼쳐기
    toggle_el = await page.query_selector('a.gKP9i[aria-expanded="false"]')
    if toggle_el:
        await toggle_el.click()
        await page.wait_for_timeout(random.randint(a=800, b=1200)) # 약간 대기
    # 서비스
    service_el = await page.query_selector('div.xPvPE')
    # 별점
    rating_el = await page.query_selector('span.PXMot.LXIwF')
    rating_text = await rating_el.text_content() if rating_el else None
    naver_rating = re.search(pattern=r"(\d)+", rating_text).group() if rating_text else "N/A"
    # 방문자 리뷰 수, 블로그 리뷰 수
    visitor_review_el = await page.query_selector('a[href*="/review/visitor"]')
    blog_review_el = await page.query_selector('a[href*="review/ugc"]')
    visitor_review_text = await visitor_review_el.text_content() if visitor_review_el else ""
    blog_review_text = await blog_review_el.text_content() if blog_review_el else ""
    visitor_review_count = int(re.sub(pattern=r"^\d+", repl="", visitor_review_text)) if visitor_review_text else 0
    blog_review_count = int(re.sub(pattern=r"^\d+", repl="", blog_review_text)) if blog_review_text else 0
    if isinstance(blog_review_count, tuple):
        blog_review_count = blog_review_count[0]
    # 배지
    badge_els = await page.query_selector_all('div.XtBbS')
    badges = []
    for el in badge_els:
        text = await el.text_content()
        if text:
            badges.append(text.strip())
    # 수집 시작
    crawled_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    info["place_id"] = place_id
    info["name"] = await name_el.text_content() if name_el else "N/A"
    info["category"] = await category_el.text_content() if category_el else "N/A"
    info["address"] = await address_el.text_content() if address_el else "N/A"
    info["opening_hours"] = await parse_opening_hours(page)
    info["services"] = await service_el.text_content() if service_el else "N/A"
    info["naver_rating"] = naver_rating
    info["visitor_review_count"] = visitor_review_count
    info["blog_review_count"] = blog_review_count
    info["badges"] = ", ".join(badges) if badges else "N/A"
    info["crawled_at"] = crawled_at
    info["adm_dong_code"] = adm_dong_code
except Exception as e:
    print(f"[{place_id}] Error crawling place info: {e}")
return info
```

```
async def fetch_info_page(page: Page, place_id: str):
    url = f"https://m.place.naver.com/restaurant/{place_id}/information"
    await page.goto(url, timeout=15000)
    await page.wait_for_load_state(state='domcontentloaded', timeout=15000)
    for _ in range(6):
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight)")
        await asyncio.sleep(0.5)
    return await safe_page_content(page, timeout=10)
```

• Crawling Code Description

run() > 2. Individual place data crawling

place_data_collector.py

fetch_review_page_and_get_reviews (page, place_id)

: Method for crawling review tab

- URL: `https://m.place.naver.com/restaurant/{place_id}/review`
- Verify ≥ 100 public receipt reviews: Stop crawling if insufficient (not saving)
 - `div.place_section_header_title` → parsing `<em class="place_section_count">`
- Per review item : `li.place_apply_pui`
 - Need to click "More" button to load 100 reviews : Click `a.fvwqf` + scroll down
- Per review data
 - Nickname : `div.pui__JiVbY3 span.pui__NMi-Dp`
 - Content : `div.pui__vn15t2 a`
 - Date : `div.pui__QKE5Pr > span.pui__gfuUIT > parse span.pui__blind`
 - Visit situation keywords : `a.pui__uqSlGI > span.pui__V8F9nN` (multiple)
 - Review keywords : `div.pui__HLNvml > span.pui__jhpEyP` (multiple)
 - Review count : `div.pui__RuLAax > span:nth-child(1)`
 - Visit frequency : `div.pui__QKE5Pr > span.pui__gfuUIT:nth-child(2)`

```
# 리뷰 크롤링
async def fetch_review_page_and_get_reviews(page, place_id): 2개의 사용 위치   ⚡ Seohyun Lee
    url = f"https://m.place.naver.com/restaurant/{place_id}/review"
    await page.goto(url)
    await page.wait_for_timeout(5000)

    MAX_REVIEWS = 100
    review_header = await page.query_selector('div.place_section_header_title')
    if review_header:
        review_html = await review_header.inner_html()
        match = re.search(pattern: r'<em class="place_section_count">(\d*)</em>', review_html)
        if match:
            total_reviews = int(match.group(1))
            if total_reviews < MAX_REVIEWS:
                print(f"❌ 리뷰 수 부족: {total_reviews}개 → 수집 제외")
                raise ValueError("리뷰 수 부족")

    # 스크롤/더보기 버튼 통해 최대한 리뷰 로딩 (최대 MAX_REVIEWS개)
    for _ in range(10): # 더보기 버튼 클릭 최대 횟수
        review_items = await page.query_selector_all("li.place_apply_pui")
        if len(review_items) >= MAX_REVIEWS:
            break

        await page.mouse.wheel(0, 5000)
        await page.wait_for_timeout(500)

    prev_count = len(review_items)
    more_btn = await page.query_selector("a.fvwqf") # 더보기 버튼
    if more_btn:
        try:
            await more_btn.click()
            await page.wait_for_timeout(2000)

            # 더보기 클릭 후 리뷰 수가 늘어났는지 확인
            for _ in range(2): # 최대 2회 재시도
                new_items = await page.query_selector_all("li.place_apply_pui")
                if len(new_items) > prev_count:
                    break
                await page.wait_for_timeout(60000) # 1분 대기
            except Exception as e:
                print(f"⚠️ 더보기 클릭 실패: {e}")
                break
        else:
            break
    await page.wait_for_timeout(3000)

    review_items = await page.query_selector_all("li.place_apply_pui")

    review_count = len(review_items)
    if review_count < MAX_REVIEWS:
        print(f"리뷰 수 부족({review_count}개), 크롤링 생략")
        return ValueError("리뷰 수 부족")
    else:
        print(f"리뷰 수집 대상: {review_count}개")

    result = []
    for r in review_items[:MAX_REVIEWS]:
        try:
            nickname_el = await r.query_selector("div.pui__JiVbY3 span.pui__uslU0d span.pui__NMi-Dp")
            content_el = await r.query_selector("div.pui__vn15t2 a")

            nickname = await nickname_el.text_content() if nickname_el else "N/A"
            content = await content_el.text_content() if content_el else "N/A"

            # 날짜 element를 모두 찾기
            date_els = await r.query_selector_all("div.pui__QKE5Pr > span.pui__gfuUIT > span.pui__blind")
            date_raw = await date_els[1].text_content() if len(date_els) > 1 else "N/A"

            # 날짜 파싱
            date = "N/A"
            if date_raw != "N/A":
                m = re.search(pattern: r"(\d{4})년 (\d{1,2})월 (\d{1,2})일", date_raw)
                if m:
                    y, mth, d = m.groups()
                    date = f"{int(y):04d}-{int(mth):02d}-{int(d):02d}"

            # 방문 상황 키워드(점심, 데이트, 바운 입장 등)
            situation_els = await r.query_selector_all("a.pui__uqSlGI > span.pui__V8F9nN")
            situations = [await el.text_content() for el in situation_els]
            situations_str = ", ".join([s.strip() for s in situations])

            # 리뷰 키워드(맛, 분위기 등)
            keyword_els = await r.query_selector_all("div.pui__HLNvml > span.pui__jhpEyP")
            keywords = [await el.text_content() for el in keyword_els]
            keywords_str = ", ".join([k.strip() for k in keywords])

            # 리뷰 개수 및 방문 차수 추출
            review_count_el = await r.query_selector("div.pui__RuLAax > span:nth-child(1)")
            visit_count_el = await r.query_selector("div.pui__QKE5Pr > span.pui__gfuUIT:nth-child(2)")
            review_count = await review_count_el.text_content() if review_count_el else "N/A"
            visit_count = await visit_count_el.text_content() if visit_count_el else "N/A"

            # 리뷰 수: "리뷰 14" → 14
            review_count = re.sub(pattern: r"[^\\d]", repl: "", review_count)
            # 방문 차수: "1번째 방문" → 1
            visit_count = re.sub(pattern: r"[^\\d]", repl: "", visit_count)

            review_data = {
                "place_id": place_id,
                "nickname": nickname.strip(),
                "content": content.strip(),
                "date": date.strip(),
                "situations": situations_str,
                "keywords": keywords_str,
                "review_count": review_count,
                "visit_count": visit_count
            }
            print(f"수집 완료: {review_data}")
            result.append(review_data)
        except Exception as e:
            print(f"[{place_id}] Error parsing review: {e}")

    return result
```

```
result = []
for r in review_items[:MAX_REVIEWS]:
    try:
        nickname_el = await r.query_selector("div.pui__JiVbY3 span.pui__uslU0d span.pui__NMi-Dp")
        content_el = await r.query_selector("div.pui__vn15t2 a")

        nickname = await nickname_el.text_content() if nickname_el else "N/A"
        content = await content_el.text_content() if content_el else "N/A"

        # 날짜 element를 모두 찾기
        date_els = await r.query_selector_all("div.pui__QKE5Pr > span.pui__gfuUIT > span.pui__blind")
        date_raw = await date_els[1].text_content() if len(date_els) > 1 else "N/A"

        # 날짜 파싱
        date = "N/A"
        if date_raw != "N/A":
            m = re.search(pattern: r"(\d{4})년 (\d{1,2})월 (\d{1,2})일", date_raw)
            if m:
                y, mth, d = m.groups()
                date = f"{int(y):04d}-{int(mth):02d}-{int(d):02d}"

        # 방문 상황 키워드(점심, 데이트, 바운 입장 등)
        situation_els = await r.query_selector_all("a.pui__uqSlGI > span.pui__V8F9nN")
        situations = [await el.text_content() for el in situation_els]
        situations_str = ", ".join([s.strip() for s in situations])

        # 리뷰 키워드(맛, 분위기 등)
        keyword_els = await r.query_selector_all("div.pui__HLNvml > span.pui__jhpEyP")
        keywords = [await el.text_content() for el in keyword_els]
        keywords_str = ", ".join([k.strip() for k in keywords])

        # 리뷰 개수 및 방문 차수 추출
        review_count_el = await r.query_selector("div.pui__RuLAax > span:nth-child(1)")
        visit_count_el = await r.query_selector("div.pui__QKE5Pr > span.pui__gfuUIT:nth-child(2)")
        review_count = await review_count_el.text_content() if review_count_el else "N/A"
        visit_count = await visit_count_el.text_content() if visit_count_el else "N/A"

        # 리뷰 수: "리뷰 14" → 14
        review_count = re.sub(pattern: r"[^\\d]", repl: "", review_count)
        # 방문 차수: "1번째 방문" → 1
        visit_count = re.sub(pattern: r"[^\\d]", repl: "", visit_count)

        review_data = {
            "place_id": place_id,
            "nickname": nickname.strip(),
            "content": content.strip(),
            "date": date.strip(),
            "situations": situations_str,
            "keywords": keywords_str,
            "review_count": review_count,
            "visit_count": visit_count
        }
        print(f"수집 완료: {review_data}")
        result.append(review_data)
    except Exception as e:
        print(f"[{place_id}] Error parsing review: {e}")

return result
```


• Crawling Code Description

2. Individual Place Data Crawling Execution > **Save**

save_data.py

1. save_place_info_json(place_info, adm_dong_code)

- Save restaurant place data as JSON, accumulated by administrative district

2. save_reviews_json(reviews, adm_dong_code)

- Save review data as JSON, accumulated by administrative district

3. save_failed_places_json(place_id, adm_dong_code)

- Save failed place IDs separately as JSON

4. save_place_raw_html(place_id, adm_dong_code, html_data)

- Save raw data per restaurant as HTML

```

save_data.py x
1  import json
2  import os
3  from typing import List, Dict
4
5  PLACE_INFO_DIR = "json_data/place_info"
6  REVIEWS_DIR = "json_data/reviews"
7  FAILED_PLACES_DIR = "json_data/failed_places"
8  RAW_DATA_DIR = "raw_data"
9
10 def _load_json_list(path: str) -> List[Dict]: 3개의 사용 위치  A: Seohyun Lee
11     """JSON 파일을 불러오거나 없으면 빈 리스트를 반환"""
12     if os.path.exists(path):
13         with open(path, "r", encoding="utf-8-sig") as f:
14             return json.load(f)
15     return []
16
17 def _save_json_list(path: str, data: List[Dict]) -> None: 3개의 사용 위치  A: Seohyun Lee
18     """리스트 데이터를 JSON으로 저장"""
19     os.makedirs(os.path.dirname(path), exist_ok=True)
20     with open(path, "w", encoding="utf-8-sig") as f:
21         json.dump(data, f, ensure_ascii=False, indent=2)
22
23 def save_place_info_json(place_info: Dict, adm_dong_code: int) -> None: 2개의 사용 위치  A: Seohyun Lee
24     """장소 정보를 JSON으로 저장"""
25     os.makedirs(PLACE_INFO_DIR, exist_ok=True)
26     path = os.path.join(PLACE_INFO_DIR, f"place_info_{adm_dong_code}.json")
27     data = _load_json_list(path)
28     data.append(place_info)
29     _save_json_list(path, data)
30
31 def save_reviews_json(reviews: List[Dict], adm_dong_code: int) -> None: 2개의 사용 위치  A: Seohyun Lee
32     """리뷰 정보를 JSON으로 저장"""
33     if not reviews:
34         return
35     os.makedirs(REVIEWS_DIR, exist_ok=True)
36     path = os.path.join(REVIEWS_DIR, f"reviews_{adm_dong_code}.json")
37     data = _load_json_list(path)
38     data.extend(reviews) # 리뷰는 리스트라 extend로 추가
39     _save_json_list(path, data)
40
41 def save_failed_places_json(place_id: str, adm_dong_code: int) -> None: 2개의 사용 위치  A: Seohyun Lee
42     """실패한 장소 정보를 JSON으로 저장"""
43     os.makedirs(FAILED_PLACES_DIR, exist_ok=True)
44     path = os.path.join(FAILED_PLACES_DIR, f"failed_places_{adm_dong_code}.json")
45     data = _load_json_list(path)
46     data.append({"adm_dong_code": adm_dong_code, "place_id": place_id})
47     _save_json_list(path, data)
48
49 def save_place_raw_html(place_id: str, adm_dong_code: int, html_data: dict) -> None: 2개의 사용 위치  A: Seohyun Lee
50     """HTML 데이터를 저장 (이미 있으면 저장하지 않음)"""
51     os.makedirs(RAW_DATA_DIR, exist_ok=True)
52     html_path = os.path.join(RAW_DATA_DIR, f"adc_{adm_dong_code}_place_rawdata_{place_id}.html")
53     # 이미 HTML 저장됨 + 스킵
54     if os.path.exists(html_path):
55         return
56     try:
57         with open(html_path, "w", encoding="utf-8-sig") as f:
58             f.write("===== HOME =====\n")
59             f.write(html_data.get("home_html") or "")
60             f.write("\n\n===== INFO =====\n")
61             f.write(html_data.get("info_html") or "")
62             f.write("\n\n===== REVIEWS =====\n")
63             f.write(html_data.get("reviews_html") or "")
64     except Exception as e:
65         print(f"[ERROR] HTML 저장 실패 (PlaceID: {place_id}) - {e}")

```

• Crawling Results

Restaurant & Reviews JSON

```

1 {
2   "place_id": "13005867",
3   "name": "온성보령",
4   "category": "죽방, 보령",
5   "address": "서울 성동구 독서당로 297-6",
6   "opening_hours": "월 10:30 - 22:20 화 10:30 - 22:20 수 정기휴무 (매주 수요일) 목 10:30 - 22:20 금 10:30 - 22:20 토 10:30 - 22:20",
7   "services": "포장, 무선 인터넷",
8   "naver_rating": "4.22",
9   "visitor_review_count": 1,
10  "blog_review_count": 546,
11  "badges": "맛있는 보령과 푸자",
12  "crawled_at": "2025-05-06",
13  "adm_dong_code": "112006",
14 },
15 {
16   "place_id": "105125450",
17   "name": "채원감자탕",
18   "category": "감자탕",
19   "address": "서울 성동구 금호",
20   "opening_hours": "수 11:00 - 22:00",
21   "services": "포장, 무선 인터넷",
22   "naver_rating": "4.47",
23   "visitor_review_count": 1,
24   "blog_review_count": 36,
25   "badges": "골골한 국물과 복숭아",
26   "crawled_at": "2025-05-06",
27   "adm_dong_code": "112006",
28 },
29 {
30   "place_id": "1171835183",
31   "name": "신금호소불암비 본점",
32   "category": "육류, 고기요리",
33   "address": "서울 성동구 무수",
34   "opening_hours": "월 11:00 - 22:00",
35   "services": "예약, 단체 이용 가능",
36   "naver_rating": "4.41",
37   "visitor_review_count": 208,
38   "blog_review_count": 208,
39   "badges": "맛있는 고기와 콩",
40   "crawled_at": "2025-05-06",
41   "adm_dong_code": "112006",
42 },
43 {
44   "place_id": "1458127314",
45   "name": "한남동감자탕 신금호",
46   "category": "감자탕",
47   "address": "서울 성동구 무수",
48   "opening_hours": "월 10:00 - 22:00",
49   "services": "단체 이용 가능",
50   "naver_rating": "N/A",
51   "visitor_review_count": 635,
52   "blog_review_count": 635,
53   "badges": "맛있는 음식과 푸자",
54   "crawled_at": "2025-05-06",
55   "adm_dong_code": "112006",
56 },
57 {
58   "place_id": "13005867",
59   "nickname": "새로움0151",
60   "content": "금남시장 내의 맛집으로 소문났다해서 가 봤어요~고기가 부드럽고, 길이 나오는 김치가 엄청 맛있네요. 굴전도 넘 맛있었어요. 멀리 국수는 국물",
61   "date": "2025-04-21",
62   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친척 형제",
63   "keywords": "음식이 맛있어요",
64   "review_count": "36",
65   "visit_count": "1",
66 },
67 {
68   "place_id": "13005867",
69   "nickname": "인생은 짧고 맛있는 건 많다",
70   "content": "나혼산 보고 있는데 맛집 맞네요. 아들이들한 보령에 한집서 푸짐하게 나온 맛갈스런 김치 조합은 역시나.. 삼겹보쌈이 제일 부드럽고 맛있",
71   "date": "2025-04-18",
72   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 가족모임, 부모님, 친척 형제",
73   "keywords": "음식이 맛있어요",
74   "review_count": "896",
75   "visit_count": "1",
76 },
77 {
78   "place_id": "13005867",
79   "nickname": "수달조아",
80   "content": "전짜 오랜만에 방문했는데 맛있는 김치와 보령이 여전히 맛있어서 좋았어요~ 삼겹살 들어간 보령보다는 기름기 적은 온성 보령이 더 담백해서 추천",
81   "date": "2025-04-03",
82   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친구",
83   "keywords": "음식이 맛있어요",
84   "review_count": "52",
85   "visit_count": "1",
86 },
87 {
88   "place_id": "13005867",
89   "nickname": "버섯미장JYP",
90   "content": "맛내냐~~",
91   "date": "2025-03-30",
92   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 데이트, 연인 배우자",
93   "keywords": "음식이 맛있어요",
94   "review_count": "1404",
95   "visit_count": "1",
96 },
97 {
98   "place_id": "13005867",
99   "nickname": "sy*****",
100  "content": "굿",
101  "date": "2025-03-29",
102  "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 일상, 연인 배우자",
103  "keywords": "집내가 좋아요",
104  "review_count": "1087",
105  "visit_count": "1",
106 },
107 {
108   "place_id": "13005867",
109   "nickname": "새로움0151",
110   "content": "금남시장 내의 맛집으로 소문났다해서 가 봤어요~고기가 부드럽고, 길이 나오는 김치가 엄청 맛있네요. 굴전도 넘 맛있었어요. 멀리 국수는 국물",
111   "date": "2025-04-21",
112   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친척 형제",
113   "keywords": "음식이 맛있어요",
114   "review_count": "36",
115   "visit_count": "1",
116 },
117 {
118   "place_id": "13005867",
119   "nickname": "인생은 짧고 맛있는 건 많다",
120   "content": "나혼산 보고 있는데 맛집 맞네요. 아들이들한 보령에 한집서 푸짐하게 나온 맛갈스런 김치 조합은 역시나.. 삼겹보쌈이 제일 부드럽고 맛있",
121   "date": "2025-04-18",
122   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 가족모임, 부모님, 친척 형제",
123   "keywords": "음식이 맛있어요",
124   "review_count": "896",
125   "visit_count": "1",
126 },
127 {
128   "place_id": "13005867",
129   "nickname": "수달조아",
130   "content": "전짜 오랜만에 방문했는데 맛있는 김치와 보령이 여전히 맛있어서 좋았어요~ 삼겹살 들어간 보령보다는 기름기 적은 온성 보령이 더 담백해서 추천",
131   "date": "2025-04-03",
132   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친구",
133   "keywords": "음식이 맛있어요",
134   "review_count": "52",
135   "visit_count": "1",
136 },
137 {
138   "place_id": "13005867",
139   "nickname": "버섯미장JYP",
140   "content": "맛내냐~~",
141   "date": "2025-03-30",
142   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 데이트, 연인 배우자",
143   "keywords": "음식이 맛있어요",
144   "review_count": "1404",
145   "visit_count": "1",
146 },
147 {
148   "place_id": "13005867",
149   "nickname": "sy*****",
150   "content": "굿",
151   "date": "2025-03-29",
152   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 일상, 연인 배우자",
153   "keywords": "집내가 좋아요",
154   "review_count": "1087",
155   "visit_count": "1",
156 },
157 {
158   "place_id": "13005867",
159   "nickname": "새로움0151",
160   "content": "금남시장 내의 맛집으로 소문났다해서 가 봤어요~고기가 부드럽고, 길이 나오는 김치가 엄청 맛있네요. 굴전도 넘 맛있었어요. 멀리 국수는 국물",
161   "date": "2025-04-21",
162   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친척 형제",
163   "keywords": "음식이 맛있어요",
164   "review_count": "36",
165   "visit_count": "1",
166 },
167 {
168   "place_id": "13005867",
169   "nickname": "인생은 짧고 맛있는 건 많다",
170   "content": "나혼산 보고 있는데 맛집 맞네요. 아들이들한 보령에 한집서 푸짐하게 나온 맛갈스런 김치 조합은 역시나.. 삼겹보쌈이 제일 부드럽고 맛있",
171   "date": "2025-04-18",
172   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 가족모임, 부모님, 친척 형제",
173   "keywords": "음식이 맛있어요",
174   "review_count": "896",
175   "visit_count": "1",
176 },
177 {
178   "place_id": "13005867",
179   "nickname": "수달조아",
180   "content": "전짜 오랜만에 방문했는데 맛있는 김치와 보령이 여전히 맛있어서 좋았어요~ 삼겹살 들어간 보령보다는 기름기 적은 온성 보령이 더 담백해서 추천",
181   "date": "2025-04-03",
182   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친구",
183   "keywords": "음식이 맛있어요",
184   "review_count": "52",
185   "visit_count": "1",
186 },
187 {
188   "place_id": "13005867",
189   "nickname": "버섯미장JYP",
190   "content": "맛내냐~~",
191   "date": "2025-03-30",
192   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 데이트, 연인 배우자",
193   "keywords": "음식이 맛있어요",
194   "review_count": "1404",
195   "visit_count": "1",
196 },
197 {
198   "place_id": "13005867",
199   "nickname": "sy*****",
200   "content": "굿",
201   "date": "2025-03-29",
202   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 일상, 연인 배우자",
203   "keywords": "집내가 좋아요",
204   "review_count": "1087",
205   "visit_count": "1",
206 },
207 {
208   "place_id": "13005867",
209   "nickname": "새로움0151",
210   "content": "금남시장 내의 맛집으로 소문났다해서 가 봤어요~고기가 부드럽고, 길이 나오는 김치가 엄청 맛있네요. 굴전도 넘 맛있었어요. 멀리 국수는 국물",
211   "date": "2025-04-21",
212   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친척 형제",
213   "keywords": "음식이 맛있어요",
214   "review_count": "36",
215   "visit_count": "1",
216 },
217 {
218   "place_id": "13005867",
219   "nickname": "인생은 짧고 맛있는 건 많다",
220   "content": "나혼산 보고 있는데 맛집 맞네요. 아들이들한 보령에 한집서 푸짐하게 나온 맛갈스런 김치 조합은 역시나.. 삼겹보쌈이 제일 부드럽고 맛있",
221   "date": "2025-04-18",
222   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 가족모임, 부모님, 친척 형제",
223   "keywords": "음식이 맛있어요",
224   "review_count": "896",
225   "visit_count": "1",
226 },
227 {
228   "place_id": "13005867",
229   "nickname": "수달조아",
230   "content": "전짜 오랜만에 방문했는데 맛있는 김치와 보령이 여전히 맛있어서 좋았어요~ 삼겹살 들어간 보령보다는 기름기 적은 온성 보령이 더 담백해서 추천",
231   "date": "2025-04-03",
232   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친구",
233   "keywords": "음식이 맛있어요",
234   "review_count": "52",
235   "visit_count": "1",
236 },
237 {
238   "place_id": "13005867",
239   "nickname": "버섯미장JYP",
240   "content": "맛내냐~~",
241   "date": "2025-03-30",
242   "situations": "점심에 방문, 예약 없이 이용,
```

Raw Data HTML

[illegible]

• Crawling Results

Save to Google Drive

- Restaurant & Reviews Data - JSON
 - Stored by administrative district code (712.4MB)
 - place_info files: 425
 - review files: 425
- Raw Data - HTML
 - Stored by restaurant (20.560GB)
 - Files: 17,157



[place_info]

이름	소유자
place_info_1111051500.json	나
place_info_1111053000.json	나

[review]

이름	소유자
reviews_1111051500.json	나
reviews_1111053000.json	나

[html]

이름	소유자
adc_1111051500_place_rawdata_11808181.html	나
adc_1111051500_place_rawdata_11828877.html	나

04

Data preprocessing and storage

- Data load

Load and use JSON files stored in shared Google Drive



crawling > json-data > place_info

... > json-data > place_info	
유형	사람
수정 날짜	출처
이름	소유자
place_info_1174070000.json	나
place_info_1174069000.json	나
place_info_1174068500.json	나
place_info_1174066000.json	나
place_info_1174065000.json	나
place_info_1174064000.json	나
place_info_1174062000.json	나
place_info_1174061000.json	나

425 JSON files

17,157 documents

crawling > json-data > reviews

... > json-data > reviews	
유형	사람
수정 날짜	출처
이름	소유자
reviews_1174070000.json	나
reviews_1174069000.json	나
reviews_1174068500.json	나
reviews_1174066000.json	나
reviews_1174065000.json	나
reviews_1174064000.json	나
reviews_1174062000.json	나
reviews_1174061000.json	나

425 JSON files

1,726,634 documents

• Data preprocessing and storage

1. Deduplicate places and clean reviews :

`deduplicate_places.py`

Remove duplicate place_ids
and clean associated reviews

2. Filter required columns :

`filter_columns.py`

Extract only required columns
from place and review data

3. Clean review text :

`clean_text.py`

Clean text by removing special characters,
emojis, NaN.

4. Extract nouns and remove stopwords :

`extract_nouns.py`

Extract nouns from reviews
and remove stopwords

5. Filter valid reviews :

`validate_review.py`

After all processing, remove short or meaningless
reviews to keep only valid reviews (≥ 5 words)

• Data Preprocessing Pipeline

```
def run_pipeline(region_csv_path: str): 1 usage  Kaeun Han
    try:
        logger.info("🔥 Step 1: Google Drive에서 데이터 불러오는 중...")
        df_place_info_raw = fetch_place_info()
        df_reviews_raw = fetch_reviews()
        logger.debug(f"📦 Place info shape: {df_place_info_raw.shape}, Reviews shape: {df_reviews_raw.shape}")
    except Exception as e:
        logger.error(f"❌ Google Drive 데이터 불러오기 실패: {e}")
        return

    try:
        logger.info("🔍 Step 2: 중복 place_id 처리 및 리뷰 필터링 중...")
        df_place_dedup, df_reviews_dedup, _ = deduplicate_places(df_place_info_raw, df_reviews_raw)
        logger.debug(f"✅ Deduplicated places: {df_place_dedup.shape}, Deduplicated reviews: {df_reviews_dedup.shape}")
    except Exception as e:
        logger.error(f"❌ 중복 제거 실패: {e}")
        return

    try:
        logger.info("📁 Step 3: 컬럼 필터링...")
        df_place_filtered, df_reviews_filtered = filter_columns(df_place_dedup, df_reviews_dedup)
        logger.debug(f"📄 Filtered place columns: {df_place_filtered.columns.tolist()}")
        logger.debug(f"📄 Filtered review columns: {df_reviews_filtered.columns.tolist()}")
    except Exception as e:
        logger.error(f"❌ 컬럼 필터링 실패: {e}")
        return
```

Load JSON files
from Google Drive



Deduplicate places
and clean reviews



Filter required columns only

• Data Preprocessing Pipeline

```
try:
    logger.info("🔧 Step 4: 리뷰 텍스트 클렌징...")
    logger.debug(f"🔍 클렌징 전 리뷰 수: {len(df_reviews_filtered)}")
    df_reviews_cleaned = clean_text(df_reviews_filtered)
    logger.debug(f"🔍 클렌징 후 리뷰 수: {len(df_reviews_cleaned)}")
    logger.debug(f"🔍 Cleaned review 예시: {df_reviews_cleaned['content'].iloc[0]}")
except Exception as e:
    logger.error(f"❌ 텍스트 클렌징 실패: {e}")
    return

try:
    logger.info("🧠 Step 5: 명사 추출 및 불용어 제거...")
    logger.debug(f"🔍 명사 추출 전 리뷰 수: {len(df_reviews_cleaned)}")
    df_reviews_nouns = extract_nouns_from_reviews(df_reviews_cleaned)
    logger.debug(f"🔍 명사 추출 후 리뷰 수: {len(df_reviews_nouns)}")
    logger.debug(f"🔍 Noun extraction 예시: {df_reviews_nouns['content_nouns'].iloc[0]}")
except Exception as e:
    logger.error(f"❌ 명사 추출 실패: {e}")
    return

try:
    logger.info("✅ Step 6: 유효 리뷰만 필터링...")
    logger.debug(f"🔍 유효성 검증 전 리뷰 수: {len(df_reviews_nouns)}")
    df_reviews_valid = validate_reviews(df_reviews_nouns)
    logger.debug(f"🔍 유효성 검증 후 리뷰 수: {len(df_reviews_valid)}")
    logger.debug(f"🔍 유효 리뷰 수: {len(df_reviews_valid)}")
except Exception as e:
    logger.error(f"❌ 유효 리뷰 필터링 실패: {e}")
    return
```

Review text cleaning



Noun extraction and stopword
removal



Valid review filtering

• Data Preprocessing Pipeline

```

try:
    logger.info("📦 Step 7: JSON 변환 중...")
    place_info_json, reviews_json = convert_to_json(df_place_filtered, df_reviews_valid)
    logger.debug(f"📄 JSON 샘플 place_info: {place_info_json[0]}")
    logger.debug(f"📄 JSON 샘플 review: {reviews_json[0]}")
except Exception as e:
    logger.error(f"❌ JSON 변환 실패: {e}")
    return

try:
    logger.info("🌐 Step 8: 지역구 컬렉션 초기화...")
    init_collections(region_csv_path)
    logger.debug(f"📄 CSV 경로: {region_csv_path}")
except Exception as e:
    logger.error(f"❌ 지역구 컬렉션 초기화 실패: {e}")
    return

try:
    logger.info("📁 Step 9: MongoDB에 데이터 적재 중...")
    insert_data(place_info_json, reviews_json)
    logger.debug("✅ 데이터 적재 성공")
except Exception as e:
    logger.error(f"❌ MongoDB 데이터 적재 실패: {e}")
    return

logger.info("🎉 모든 데이터 파이프라인 완료 및 MongoDB 저장 완료!")

# 실행 (python pipeline.py)
if __name__ == "__main__":
    run_pipeline("data/adm_dong_list.csv")

```

Convert dataframe to JSON



Create MongoDB region collection



Load place and review data
into MongoDB

Complete !

• Data Preprocessing Code

1. Deduplicate places and clean reviews

deduplicate_places.py

```
# preprocess/deduplicate_places.py
import pandas as pd

def deduplicate_places(df_place_info: pd.DataFrame, df_reviews: pd.DataFrame): 4 usages  Kaeun Han
    # 1. 중복 place_id 찾기
    duplicated_place_ids = df_place_info[df_place_info.duplicated(subset="place_id", keep=False)]["place_id"].unique().tolist()

    # 2. adm_dong_code 기준으로 정렬 후, 중복 제거 (adm_dong_code가 가장 작은 것만 남김)
    df_place_info_sorted = df_place_info.sort_values("adm_dong_code")
    df_place_info_dedup = df_place_info_sorted.drop_duplicates(subset="place_id", keep="first")

    # 3. reviews: dedup된 place_id 중 중복이었던 place_id에 대해 각각 100개씩만 남기기
    df_reviews_filtered = df_reviews[df_reviews["place_id"].isin(duplicated_place_ids)].copy()
    df_reviews_limited = (
        df_reviews_filtered
        .groupby("place_id")
        .head(100)
        .reset_index(drop=True)
    )

    # 4. 중복이 아니었던 place_id 리뷰 전부 유지
    non_duplicated_place_ids = set(df_place_info_dedup["place_id"]) - set(duplicated_place_ids)
    df_reviews_remaining = df_reviews[df_reviews["place_id"].isin(non_duplicated_place_ids)]

    # 5. 최종 리뷰 데이터 합치기
    df_reviews_final = pd.concat(objs=[df_reviews_limited, df_reviews_remaining], ignore_index=True)

    return df_place_info_dedup.reset_index(drop=True), df_reviews_final.reset_index(drop=True), duplicated_place_ids
```

Find duplicate place_ids



Keep the one with smallest
adm_dong_code



Remove corresponding duplicates
from reviews

- **Data Preprocessing Code**

2. Filter required columns

filter_columns.py

```
import pandas as pd

def filter_columns(df_place_info: pd.DataFrame, df_reviews: pd.DataFrame): 4 usages  👤 RyeongahLim
    df_place_filtered = df_place_info[[
        "place_id", "adm_dong_code", "name", "category", "address", "opening_hours", "naver_rating"
    ]].copy()

    df_reviews_filtered = df_reviews[["place_id", "visit_count", "content"]].copy()

    return df_place_filtered, df_reviews_filtered
```

Columns used for the analysis -
place_info: 7 columns,
reviews : 3 columns

• Data Preprocessing Code

3. Clean review text

clean_text.py

```
import pandas as pd
import re

def clean_text(df_reviews: pd.DataFrame): 4 usages Kaeun Han +1
    # 1. content 컬럼의 결측치 먼저 제거
    df_reviews = df_reviews.dropna(subset=["content"])

    # 2. 특수문자, 이모지 제거
    df_reviews.loc[:, "content"] = df_reviews["content"].apply(
        lambda x: re.sub(pattern: r"[^가-힣a-zA-Z0-9\s]", repl: "", str(x))
    )

    # 3. 공백만 남은 리뷰 제거 → 이건 "내용이 없다"는 거니까 제거
    df_reviews = df_reviews[df_reviews["content"].str.strip() != ""]

    # 4. 혹시 모르니 다시 한 번 결측치 제거 (위 처리로 인해 생겼을 수도 있는 NaN 방지)
    df_reviews = df_reviews.dropna(subset=["content"])

    return df_reviews.reset_index(drop=True)
```

Remove data with missing reviews



Remove special characters
and emojis



Remove reviews with only whitespace (no
content)



Final removal of missing values
after all steps

• Data Preprocessing Code

4. Extract nouns and remove stopwords

extract_nouns.py

```
import os
from konlpy.tag import Okt
from utils.logger import get_logger

logger = get_logger("extract_nouns")

okt = Okt() #형태소 분석기 for 명사추출

# korean_stopwords.txt 파일 경로 설정
STOPWORDS_PATH = os.path.join(os.path.dirname(__file__), "korean_stopwords.txt")

def load_stopwords(): 1 usage  ⤴ RyeongahLim
    try:
        with open(STOPWORDS_PATH, "r", encoding="utf-8") as f:
            stopwords = set(line.strip() for line in f if line.strip())
            return stopwords
    except FileNotFoundError:
        logger.error(f"Stopwords file not found at {STOPWORDS_PATH}")
        return set()

def extract_nouns_from_text(text, stopwords): 1 usage  ⤴ RyeongahLim +1
    if not isinstance(text, str):
        return []

    nouns = okt.nouns(text)
    filtered = [word for word in nouns if word not in stopwords and len(word) > 1]
    return filtered
```

Okt: Korean Morphological Analyzer

Extract nouns using okt.nouns(text)

korean_stopwords.txt : File for handling Korean stopwords

```
def extract_nouns_from_reviews(df_reviews): 4 usages  ⤴ Kaeun Han +1
    logger.info("명사 추출 및 불용어 제거 시작")
    stopwords = load_stopwords()
    total = len(df_reviews)

    content_nouns = []

    for idx, row in df_reviews.iterrows():
        text = row["content"]
        nouns = extract_nouns_from_text(text, stopwords)
        content_nouns.append(nouns)

        # 💡 중간 진행상황 로그 (1000개 단위로)
        if (idx + 1) % 1000 == 0 or (idx + 1) == total:
            logger.debug(f"🔄 명사 추출 진행 중: {idx + 1}/{total}개 완료")

    df_reviews["content_nouns"] = content_nouns
    logger.info("명사 추출 및 불용어 제거 완료")
    return df_reviews
```

- **Data Preprocessing Code**

5. Valid review filtering

valid_reviews.py

```
import pandas as pd

def validate_reviews(df_reviews: pd.DataFrame): 4 usages  👤 RyeongahLim
    df_valid = df_reviews[df_reviews["content_nouns"].apply(lambda x: len(x) > 4)]
    return df_valid.reset_index(drop=True)
```

After completing all preprocessing,
we retained only review data with 5 or more nouns
(= reviews that can define the situation adequately.)

• Data Preprocessing Results

```

- 📦 Place info shape: (17157, 12), Reviews shape: (1726634, 8)
- 🔍 Step 2: 중복 place_id 처리 및 리뷰 필터링 중...
- ✂️ Deduplicated places: (8758, 12), Deduplicated reviews: (881678, 8)
- 📁 Step 3: 컬럼 필터링...
- 📄 Filtered place columns: ['place_id', 'adm_dong_code', 'name', 'category', 'address', 'opening_hours', 'naver_rating']
- 📄 Filtered review columns: ['place_id', 'visit_count', 'content']

```

Json File Data > (17157, 12)(place_info), (1726634, 8)(reviews)

Deduplicate places and clean reviews >> (8739, 12)(place_info), (881678, 8)(reviews)

Column filtering >> (8739, 7)(place_info), (881678, 3)(reviews)

```

- 🌱 Step 4: 리뷰 텍스트 클렌징...
- 📄 클렌징 전 리뷰 수: 881678
- 📄 클렌징 후 리뷰 수: 826803
- ✂️ Cleaned review 예시: 오랜만에 파스타 맛집을 찾은거 같아 너무 좋았어요 일단 시금치 파스타는 바질과 시금치 향이 진해 너무 잘먹었습니다
  습니다문어 샐러드도 스타터로 입맛을 돋군게 딱이었습니다너무 오랜만에 잘 먹었습니다
- 🧠 Step 5: 명사 추출 및 불용어 제거...
- 📄 명사 추출 전 리뷰 수: 826803
  명사 추출 및 불용어 제거 시작

```

Review text cleaning >> 826730(reviews)

Noun extraction and stopword removal

• Data Preprocessing Results

```
✅ Step 6: 유효 리뷰만 필터링...  
- 📄 유효성 검증 전 리뷰 수: 826803  
- 📄 유효성 검증 후 리뷰 수: 342873  
- ✏ 유효 리뷰 수: 342873
```

Valid review filtering (≥ 5 nouns) >> 342873

```
📦 Step 7: JSON 변환 중...  
- 📄 JSON 샘플 place_info: {'place_id': 36819674, 'adm_dong_code': 1111051500.0, 'name': '준수방키친', 'category': '이탈리아음식', 'ad  
친', 'opening_hours': '월 11:30 - 21:30, 15:00 - 17:00 브레이크타임, 14:00, 20:30 라스트오더 화 11:30 - 21:30, 15:00 - 17:00 브레이크  
이크타임, 14:00, 20:30 라스트오더 목 11:30 - 21:30, 15:00 - 17:00 브레이크타임, 14:00, 20:30 라스트오더 금 11:30 - 21:30, 15:00 - 17:0  
20:30 라스트오더 일 11:30 - 20:30, 19:30 라스트오더', 'naver_rating': 4.43}  
- 📄 JSON 샘플 review: {'place_id': 1534186654, 'visit_count': 1.0, 'content_nouns': ['파스타', '맛집', '시금치', '파스타', '바질', '사  
리', '블루', '치즈', '소스', '기분', '문어', '샐러드', '스타', '입맛']}  
🌐 Step 8: 지역구 컬렉션 초기화...  
- 📁 CSV 경로: data/adm_dong_list.csv  
📧 Step 9: MongoDB에 데이터 적재 중...  
- ✅ 데이터 적재 성공  
🎉 모든 데이터 파이프라인 완료 및 MongoDB 저장 완료!
```

MongoDB insertion completed

- DB information



de25proejct1DB +		
de25proejct1db.twmfiwj.mongodb.net > de25proejct1DB		
Sort by Collection Name ▾ 1=		
place_info		
Storage size: 1.16 MB	Documents: 8.8 K	Avg. document size: 446.00 B
region_map		
Storage size: 28.67 kB	Documents: 425	Avg. document size: 95.00 B
reviews		
Storage size: 72.68 MB	Documents: 343 K	Avg. document size: 505.00 B

Database :
de25project1DB

Collection 1 : place_info
Collection 2 : region_map
Collection 3 : reviews

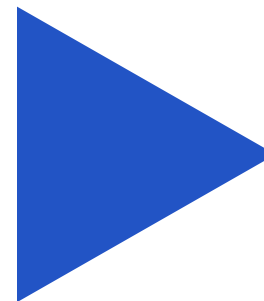
• DB information

For **region_map**,

added a CSV containing Seoul metropolitan administrative district information as a single collection

	adm_dong_code	city	district	neighborhood
1				
2	1111051500	서울특별시	종로구	청운효자동
3	1111053000	서울특별시	종로구	사직동
4	1111054000	서울특별시	종로구	삼청동
5	1111055000	서울특별시	종로구	부암동
6	1111056000	서울특별시	종로구	평창동
7	1111057000	서울특별시	종로구	무악동
8	1111058000	서울특별시	종로구	교남동
9	1111060000	서울특별시	종로구	가회동
10	1111061500	서울특별시	종로구	종로1.2.3.4가동
11	1111063000	서울특별시	종로구	종로5.6가동
12	1111064000	서울특별시	종로구	이화동
13	1111065000	서울특별시	종로구	혜화동
14	1111067000	서울특별시	종로구	창신제1동
15	1111068000	서울특별시	종로구	창신제2동

adm_dong_list.csv



de25proejct1db.twmfiwj.mongodb.net > de25proejct1DB > region_map

Documents 425 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

+ ADD DATA EXPORT DATA UPDATE DELETE

```

_id: ObjectId('67fbc1e7c3fae86e7548e50f')
adm_dong_code: 1111051500
district: "종로구"
neighborhood: "청운효자동"

_id: ObjectId('67fbc1e7c3fae86e7548e512')
adm_dong_code: 1111055000
district: "종로구"
neighborhood: "부암동"

_id: ObjectId('67fbc1e7c3fae86e7548e518')
adm_dong_code: 1111063000
district: "종로구"
neighborhood: "종로5.6가동"

_id: ObjectId('67fbc1e7c3fae86e7548e51e')
adm_dong_code: 1111070000
district: "종로구"
neighborhood: "송인제1동"

```

collection : region_map

- Final DB information

place_info

<Fields>

_id(ObjectId)
place_id(INT32)
adm_dong_code(Double)
name(String)
category(String)
address(String)
opening_hours(String)
naver_rating(Double)

Collection containing
restaurant information

region_map

<Fields>

_id(ObjectId)
adm_dong_code(INT32)
district(String)
neighborhood(String)

Collection containing
distinct information

reviews

<Fields>

_id(ObjectId)
place_id(INT32)
visit_count(Double)
content_nouns(Array)
situation_definition(String)
situation_cluster(INT32)
cluster_name(String)

Collection containing
reviews per restaurant

- **MongoDB Index Optimization**

Added indexes to enhance search performance

Collection	Index field	Purpose
place_info	adm_dong_code	Accelerate search based on administrative district
place_info	place_id	Accelerate search by place_id
reviews	place_id, situation_cluster	Accelerate review aggregation by cluster and restaurant

05

Data Analysis (ML)

• LLM Situation Definition Generating Pipeline

1. Situation definition generation: situation_definition field

Through **Claude 3.5 Haiku**, generate situation definition sentences for each review (based on noun list format)

2. Situation clustering:

Cluster situation definition sentences using **KoSBERT** + **K-means clustering (k=20)** situation_cluster field

3. Cluster name generation:

Use LLM to generate representative situation names for each cluster cluster_name field

4. Database update::

Map situation cluster information to each restaurant review and **update MongoDB**

- **LLM decision process (Claude)**

Consideration: While processing large-scale review data, it's important to generate situation definitions that are natural in context and plausible

OpenAI GPT 4o-mini

Google Gemini pro

Hugging Face Models

PracticeLLM/
Custom-KoLLM-13B-v7

Claude 3.5 Haiku



**Cost + speed
consideration**

- **KoSBERT Model**

KoSBERT info : <https://huggingface.co/jhgan/ko-sbert-nli>

Model that fine-tuned Sentence-BERT structure **sentence embeddings to Korean**

Utilized for similarity calculation to cluster **situation definition sentences**

• Claude 3.5 Haiku from Antrophic

Credit balance

Your credit balance will be consumed with API, Claude Code and Workbench usage. You can buy credits directly or set up auto-reload thresholds.

AI

US\$19.42

Remaining Balance

Charged to

link

Link by Stripe↻

Buy credits

⊗

Auto reload is disabled. Enable auto reload to avoid API interruptions when credits are fully spent.

Edit

Charge credits upfront and can make API requests within that amount

API keys 1

API keys are owned by workspaces and remain active even after the creator is removed

KEY	WORKSPACE	CREATED BY	CREATED AT ^
deproject1-onbo... sk-ant-api03-min...SAAA	Default ⓘ	DEproject1 de25project1@gmail.com	Apr 20, 2025

Apr 21, 2025	Credit grant	Expiring 4/22/2026	US\$44.00
Apr 20, 2025	Credit grant	Expiring 4/21/2026	US\$38.50
Apr 20, 2025	Credit grant	Expiring 4/21/2026	US\$5.50

- **Prompt engineering process**

Through multiple attempts, we continuously challenged ourselves to **reduce input token counts** while ensuring results came out as desired

Generating situation definitions for each review was completed with the final prompt after **6 attempts**

Generating representative situation names for each cluster was completed with the final prompt after **2n attempts**

Prompts and results for each attempt are recorded regularly in **Notion**

• Prompt engineering process

1차 시도

```
# prompt
prompt = (
    "아래 키워드를 참고해서, 어떤 상황에 이런 장소를 추천할 수 있을지 한 문장:
    "문장은 '~할 때'로 끝나는 형식으로 작성해줘.\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "상황정의:"
)
return prompt
```

- 프롬프트에 맞지 않음. ~때로 끝나지 않음.
- 응답시간, 처리시간이 중복으로 뜸.
- 응답과 상황정의가 중복으로 뜸.

2차 시도

```
# prompt
prompt = (
    "다음 키워드를 참고해서, 어떤 상황에 이런 장소를 추천할 수 있을지 한 문장으로 정의해
    "문장은 반드시 '~할 때'로 끝나야 해. 예: '가족과 함께 편안한 식사를 즐기고 싶을 때'
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

- 처리 시간이 많이 늘어남.

3차 시도

• 처리 시간은 줄었으나, ~때, ~때가 두번씩 나오는 경우가 있음. 한 문장이라고 했음에도 불구하고.

Python ▾

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 말해줘.\n"
    "문장은 '~할 때'로 끝내.\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

4차 시도 : 더 명확하게 '하나만', '한 문장'을 강조

- 나름 괜찮음. 처리시간도 그렇고. 그렇지만, 문장이 조금 긴 게 문제.

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 말해줘.\n"
    "문장은 '~할 때'로 끝내.**문장은 하나만 출력해.**\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

5차 시도

- “~레스토랑입니다.”라고 나오는 경우가 생김. 도대체 왜 내 프롬프트를 무시하는가.

```
# prompt
prompt = (
    "다음 키워드를 참고해, 이 장소를 어떤 상황에서 추천할 수 있을지 **간결한
    "반드시 '~할 때'로 끝나야 해. **하나의 상황만 말해줘.**\n\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

★ 6차 시도 : 4차에서 “간결하게”라는 표현만 추가. (완성 !!) ★

- 문장도 짧게 나오고, '~할 때'로 나옴.

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 간결하게 말해줘.\n"
    "문장은 '~할 때'로 끝내.**문장은 하나만 출력해.**\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```


- Prompt decision

```
@staticmethod 1 usage  👤 Kaeun Han
def build_prompt(content_nouns: List[str]) -> str:
    prompt = (
        "다음 키워드로 유추되는 음식점 추천 상황을 한 문장으로 간결하게 말해줘.\n"
        "문장은 '~할 때'로 끝내. 문장은 하나만 출력해.\n"
        f"키워드: {' '.join(content_nouns)}\n"
        "상황 정의:"
    )
    return prompt
```

Step1.

Prompt for
generating situation
definitions for each
review

• Prompt decision

```
class ClaudeAPIWrapper: 2 usages  Kaeun Han

def generate_label(self, situation_definitions_batch): 1 usage  Kaeun Han
    joined_text = "\n".join(f"- {s}" for s in situation_definitions_batch)

    prompt = (
        "다음 문장들을 보고 사용자가 어떤 상황일 때 해당 음식점을 방문하는 것이 좋을지, 하나의 주제 문장을 만들어줘.\n"
        "\n"
        "조건:\n"
        "- 문장은 반드시 '~할 때'로 끝내.\n"
        "- 음식 종류나 메뉴 위주로 작성하지마. 대신 기분, 추천 상황, 목적 같은 특징은 살려서 표현해줘.\n"
        "- 구체적인 욕구나 활동(예: 스트레스 풀고 싶을 때, 기름진 음식을 즐기고 싶을 때)로 표현해줘.\n"
        "- 문장 속 단어를 그대로 나열하는 식이 아니었으면 좋겠어.\n"
        "- 하나의 공통된 패턴과 의미를 반영해.\n"
        "\n"
        "예시:\n"
        "- 친구랑 수다 떨며 편하게 모이고 싶을 때\n"
        "- 비 오는 날 따뜻한 국물 요리를 즐기고 싶을 때\n"
        "\n"
        f"{joined_text}\n\n대표 문장:"
    )
```

Step3.

Prompt for
generating
representative
situation names for
each cluster

• Challenges: API Token Limit Resolution

Step1.

Generating situation definitions
for each review

Large number of reviews.
Small token count per review.



asyncio asynchronous
setting batch size,
concurrency_limit,
max_rps

Step3.

Generating representative
situation names for each cluster

Small number of clusters.
Large token count per cluster.



Synchronous processing
batch size of situation definitions
belonging to each cluster

• KoSBERT + clustering pipeline

```
# 3. 전체 실행 흐름
if __name__ == "__main__":
    # (1) 로딩
    print("🚀 merged_vectors.json 로딩 중...")
    ids, embeddings = load_vectors_from_merged_json("./step2_clustering/merged_vectors.json")
    print(f"✅ 벡터 수: {len(ids)}")

    # (2) KMeans 클러스터링
    print("🔍 KMeans 클러스터링 중...")
    kmeans = KMeans(n_clusters=20, random_state=42, n_init="auto")
    labels = kmeans.fit_predict(embeddings)

    id_to_cluster = {id_: int(cluster) for id_, cluster in zip(ids, labels)}

    # (3) 품질 평가
    print(f"📊 관성(Inertia): {kmeans.inertia_:.2f}")
    sil_score = silhouette_score(embeddings, labels)
    print(f"📊 실루엣 점수: {sil_score:.4f}")

    # (4) MongoDB 업로드
    print("📡 MongoDB에 클러스터 결과 업로드 중...")
    upload_cluster_results_to_mongo(id_to_cluster)
```

After loading the KoSBERT model,
save the merged_vectors.json file

Load merged_vectors.json file

KMeans Clustering(k=20)

Upload clustering results to MongoDB
(situation_cluster field)

• 20 Representative Situations

- 0 When you crave spicy food to relieve stress
- 1 When looking for a casual restaurant for solo dining
- 2 When you're someone who feels it's incomplete without meat in your meal
- 3 When you need a restaurant with convenient delivery or takeout
- 4 When you want to enjoy gopchang and various organ dishes
- 5 When you want to warm your body with flavorful broth
- 6 When you want to spend quality time with friends or family
- 7 When you want a pleasant meal experience with kind service
- 8 When you want to taste home-cooked style meals made with care
- 9 When you want to enjoy authentic Japanese cuisine

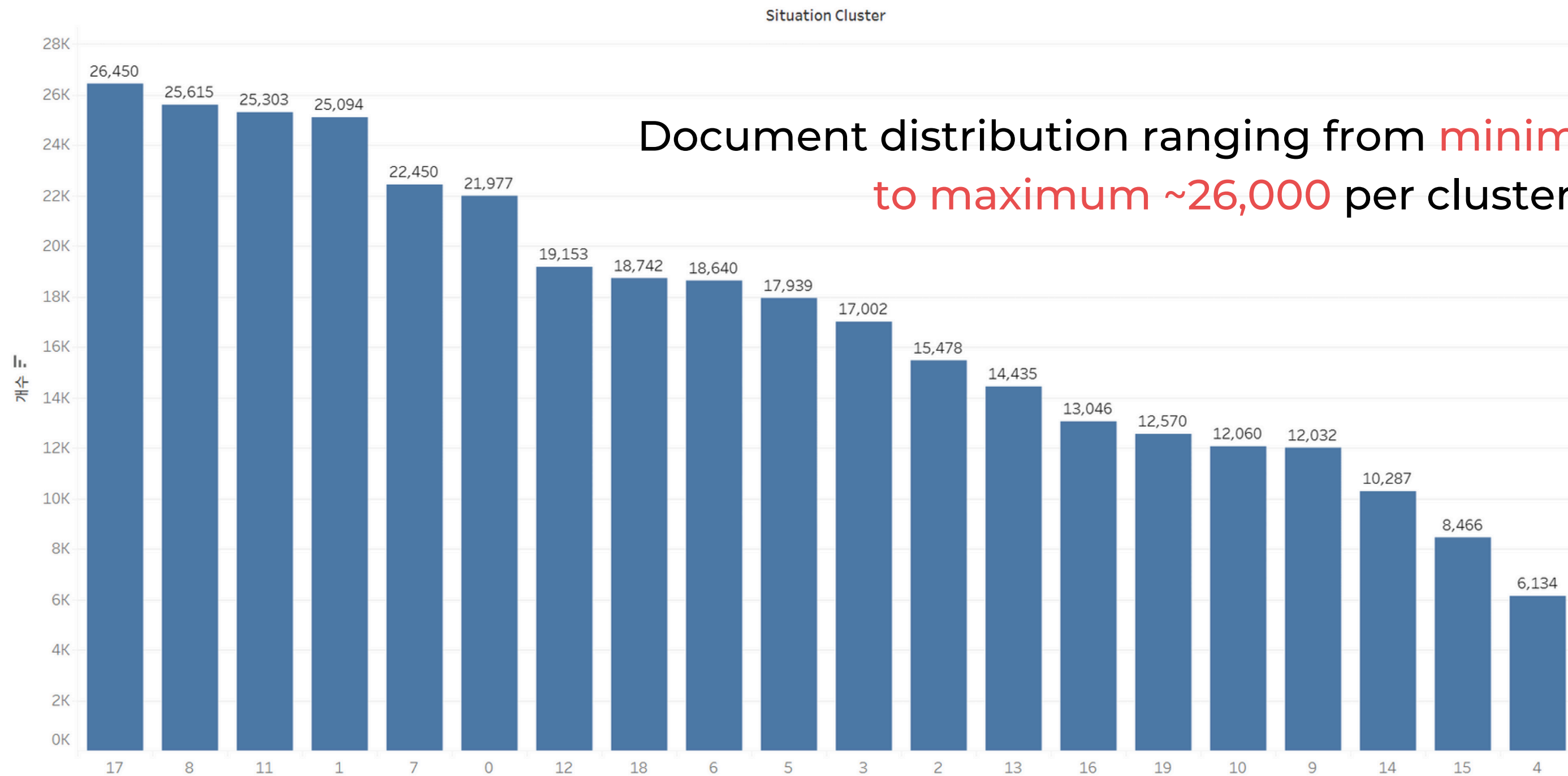
• 20 Representative Situations

- 10 When looking for a bar with delicious snacks
- 11 When you want a meal with satisfying meat quality and taste
- 12 When looking for a restaurant to visit with family on weekends
- 13 When you want to treat with Western cuisine on a date
- 14 When you want quality service for a special occasion
- 15 When looking for a Chinese restaurant with diverse menu and nice atmosphere
- 16 When you want to enjoy beer and chicken with friends casually
- 17 When you want to meet acquaintances at a good value restaurant
- 18 When you want to dine at a comfortable, relaxed place to unwind
- 19 When you want to enjoy fresh seafood dishes at a reasonable price

- 20 Representative Situations

Cluster document count confirmation chart

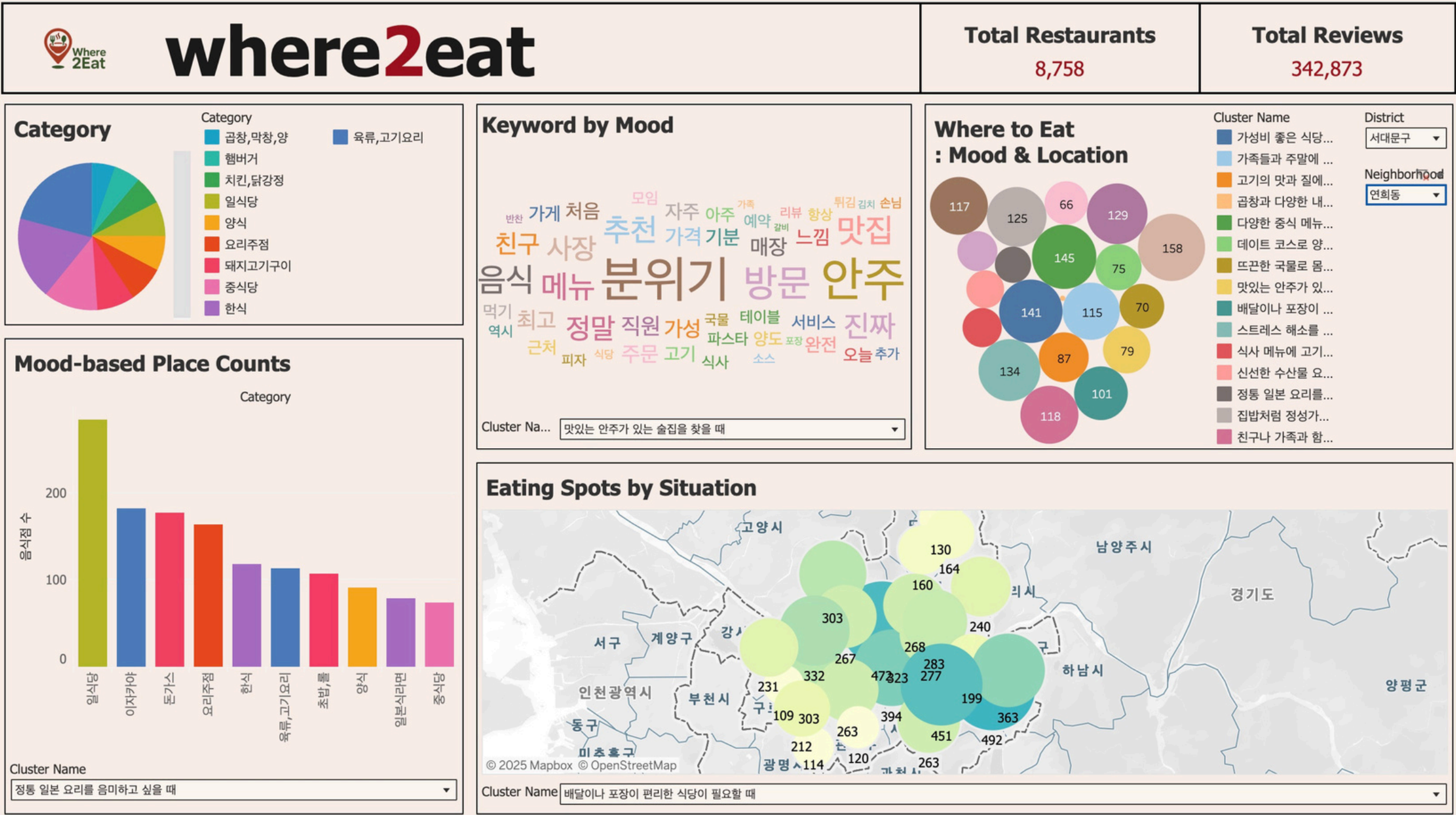
클러스터별 document 개수



06

Data visualization

• Data visualization (with Tableau)



- Total Restaurants
 - total number of restaurants
- Total Reviews
 - total number of reviews
- Category
 - Number of restaurants by category
- Keyword by Mood
 - keyword word cloud by situation
- Where to Eat: Mood & Location
 - bubble chart of situations by region
- Mood-based Place Counts
 - Bar chart of category counts by situation
- Eating Spots by Situation
 - Map of situation frequency

07

Service implementation

- Site design (with Figma)



Main page

Screen for selecting desired district



Loading page

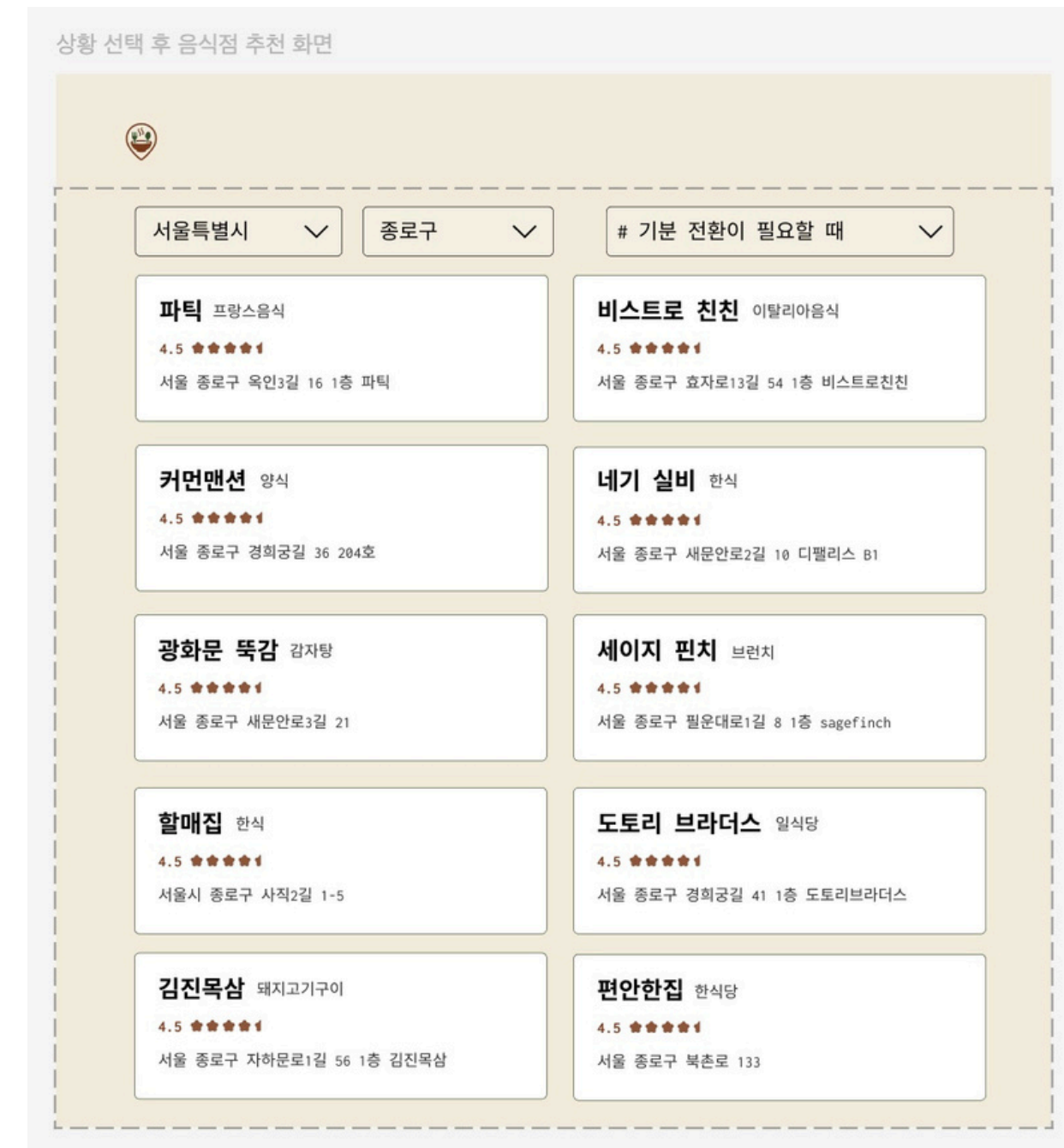
Loading screen going to bubble chart
for that district

- Site design (with Figma)



Selection page

Screen for selecting from **bubble chart** based on situation



Recommendation page

Screen showing restaurant recommendations

• Website development

Frontend : React(JS), Vite, D3.js, React Router DOM

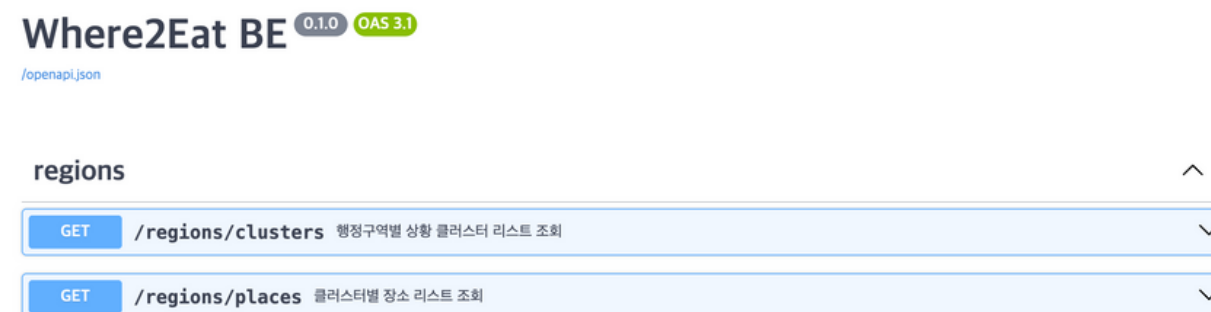
- GitHub: <https://github.com/DE-Project1/situation-restaurant-ui-react>
- Published website : <https://scintillating-sable-e44ff7.netlify.app/>



D3.js: JS-based data visualization library → **bubble chart** development

Backend : FastAPI(Python), Uvicorn, Pymongo, AWS EC2, Docker, GitHub Actions

- GitHub: <https://github.com/DE-Project1/Where2Eat-BE>
- Server API : <https://api.where2eat.r-e.kr/docs>



Developed 2 APIs connecting to MongoDB and delivering data to FE

- Results



Website logo creation with GPT

A screenshot of the 'Where 2Eat' website interface. The header shows the logo. The main content area has the title '오늘의 상황, 오늘의 맛집' (Today's Situation, Today's Delicious Place) and statistics: '서울특별시 342,873개의 식당 데이터 수' (342,873 restaurant data points in Seoul) and '수집된 총 리뷰 수 : 1,726,634개' (Total collected reviews: 1,726,634). A red line of text says '상황에 따라, 기분에 따라, 나만의 맛집을 만나보세요!' (Find your own delicious place according to the situation and mood!). On the right is an illustration of a bowl of food and a bottle. At the bottom is a search bar with the prompt '먼저 지역을 선택해 주세요. 가까운 곳부터 시작해볼까요?' (Please select a region first. Let's start from the nearest place?). The search bar contains two dropdown menus: '서울특별시' (Seoul) and '지역구' (District), followed by a '선택' (Select) button.

[Where2Eat website connection link](#)

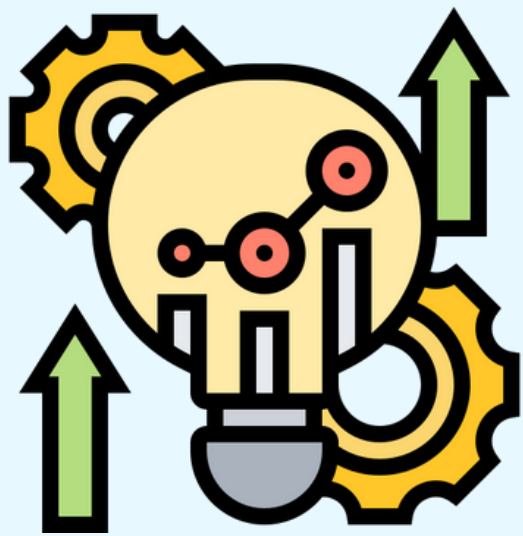
- **Service demo**



Website Demo

Since the server is currently offline,
a demo video will be shown instead.

• Further Developments



1. Expand user-personalized recommendation capability

- Utilize situation definitions from each review directly without clustering

2. Expand service scope

- Expand service from Seoul → nationwide
- Provide timely recommendations with real-time information updates

Thank You